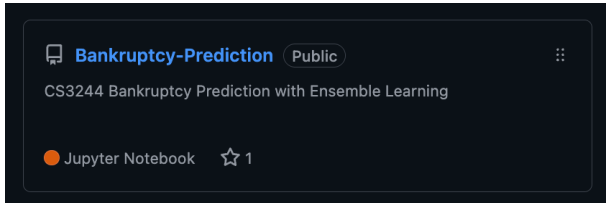
MLOps for Traditional Machine Learning

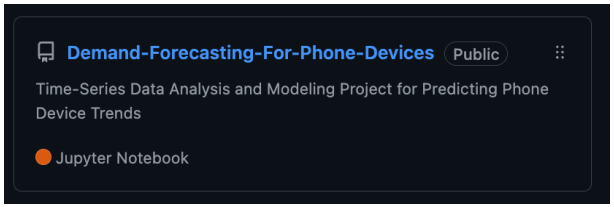
DSA4288 Honours Project in Data Science and Analytics

By: Chin Sek Yi
Supervisor: Prof. Markus Kirchberg

How many of you have built an amazing ML model

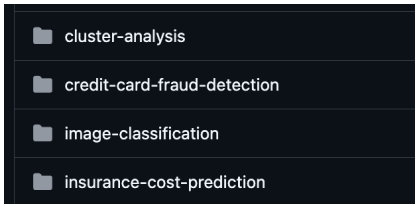
that never saw production?

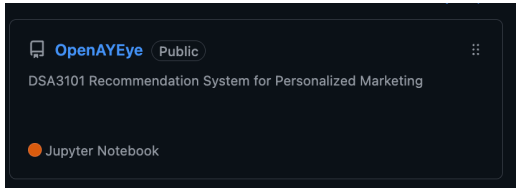


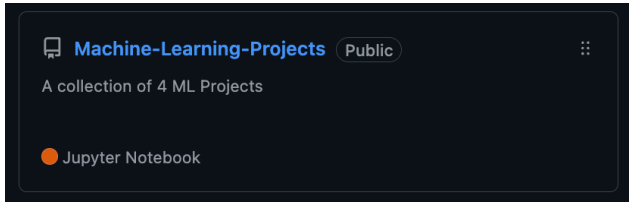


How many of you have built an amazing ML model

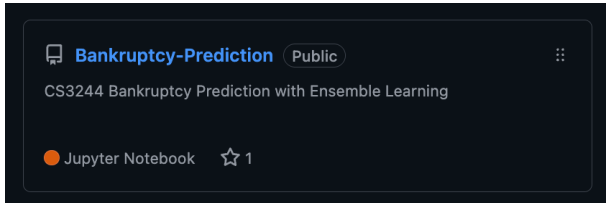
that never saw production?

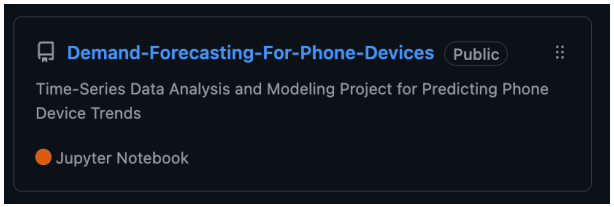




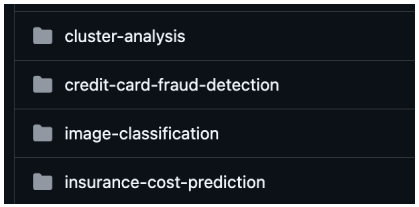


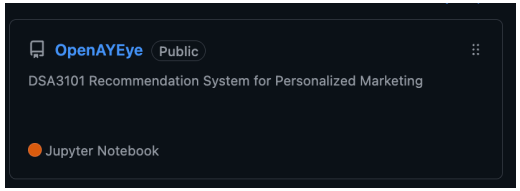
Problem Statement

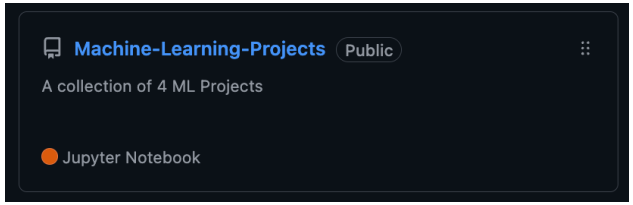




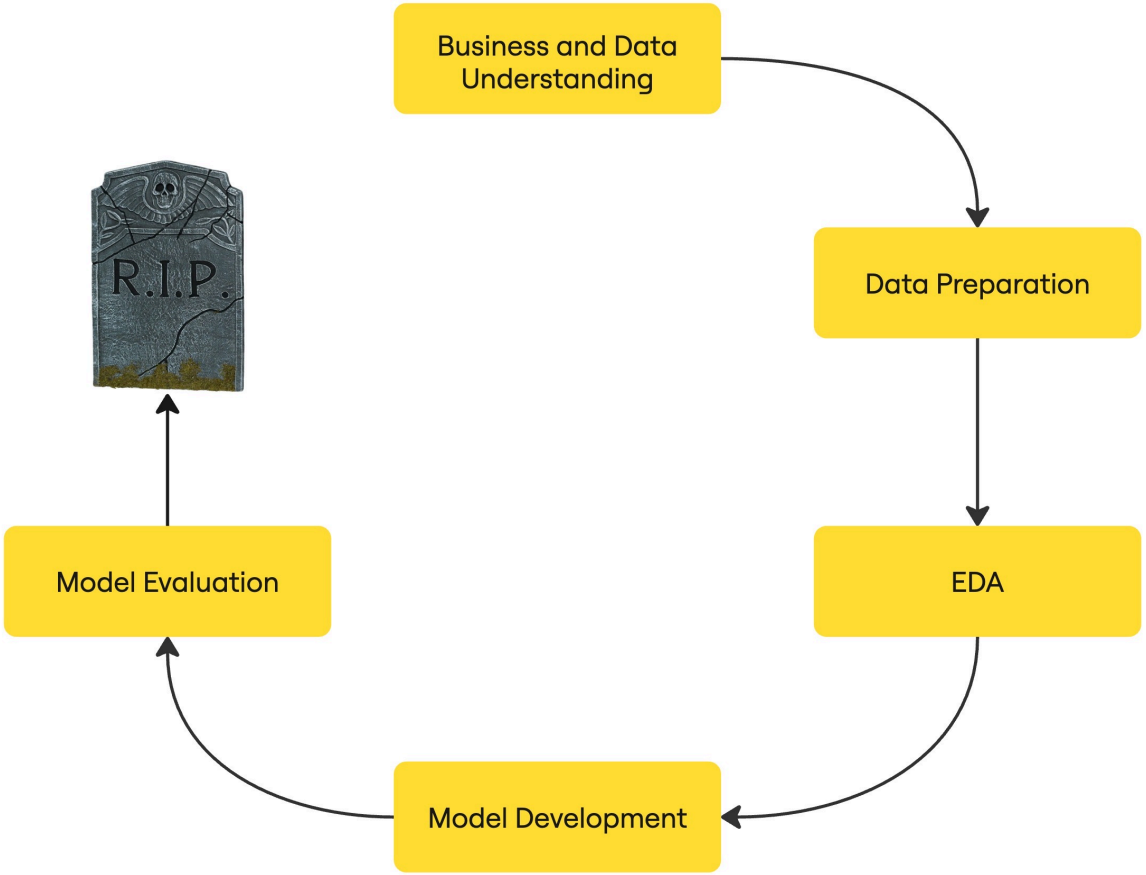
7+ ML projects, 0 in production !







Problem Statement



Objective

Machine Learning Operations

(MLOps)

Roadmap

1. MLOps Motivation
2. System Overview & Use Case — Building the Foundation
3. Pipeline Component — The Data & Model Pipeline
4. Infrastructure & Deployment— Scaling Across Environments

Demo

Roadmap

1. MLOps Motivation 

2. System Overview & Use Case — Building the Foundation

3. Pipeline Component — The Data & Model Pipeline

4. Infrastructure & Deployment— Scaling Across Environments

Demo

1.1 What is MLOps? (Machine Learning Operations)

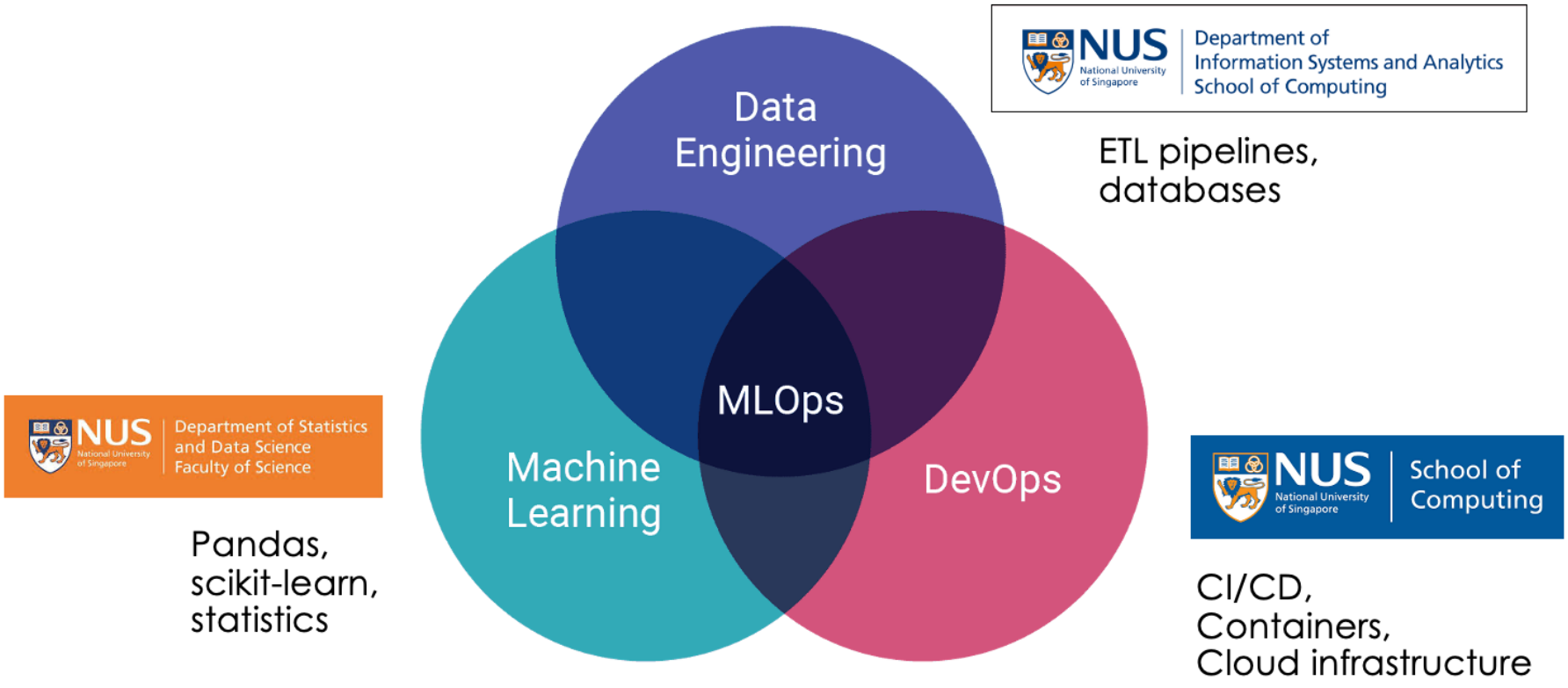
Automates and standardizes the entire ML lifecycle—from development to deployment and monitoring.

Key Features:

- End-to-end automation
- CI/CD for ML models
- Testing & monitoring
- DevOps for ML
- Reliable, production-ready systems

ML models are only as good as the systems that support them

1.2 The Skills Gap: Why Universities Don't Teach MLOps



1.3 MLOps: Bridging Two Worlds

Traditional Data Science Workflow

- Manual experimental tracking
- Jupyter notebook development
- Offline training and evaluation
- Individual researcher workflow

DevOps Workflow

- Development → Staging → Production
- CI/CD automation
- Code reviews & Pull requests
- Containerisation and Cloud infrastructure

MLOps = Both Combined

Experimental rigor + Production reliability = Production-ready ML systems

Roadmap

1. MLOps Motivation ✓

2. System Overview & Use Case — Building the Foundation

3. Pipeline Component — The Data & Model Pipeline

4. Infrastructure & Deployment— Scaling Across Environments

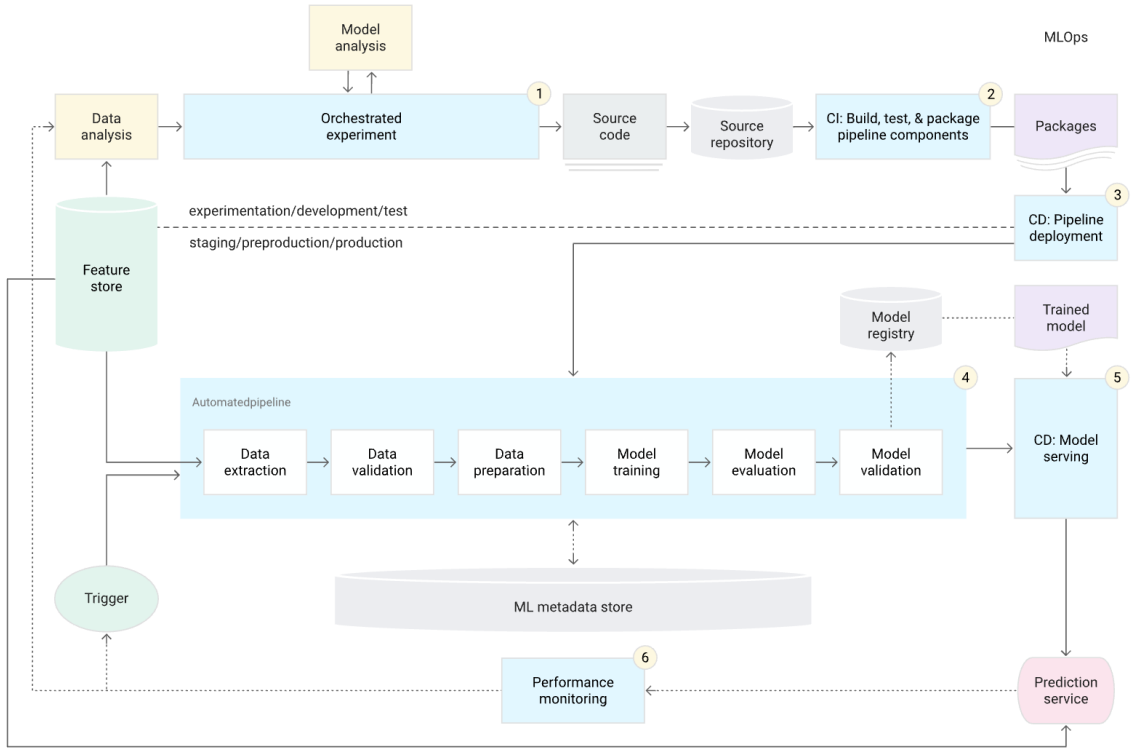
Demo

Roadmap

1. MLOps Motivation
2. System Overview & Use Case — Building the Foundation 
3. Pipeline Component — The Data & Model Pipeline
4. Infrastructure & Deployment— Scaling Across Environments

Demo

2.1 Gold Standard: MLOPs Architecture diagram (Google Cloud)



Source: Google cloud

Business use case?

2.2 Why Fraud Detection?

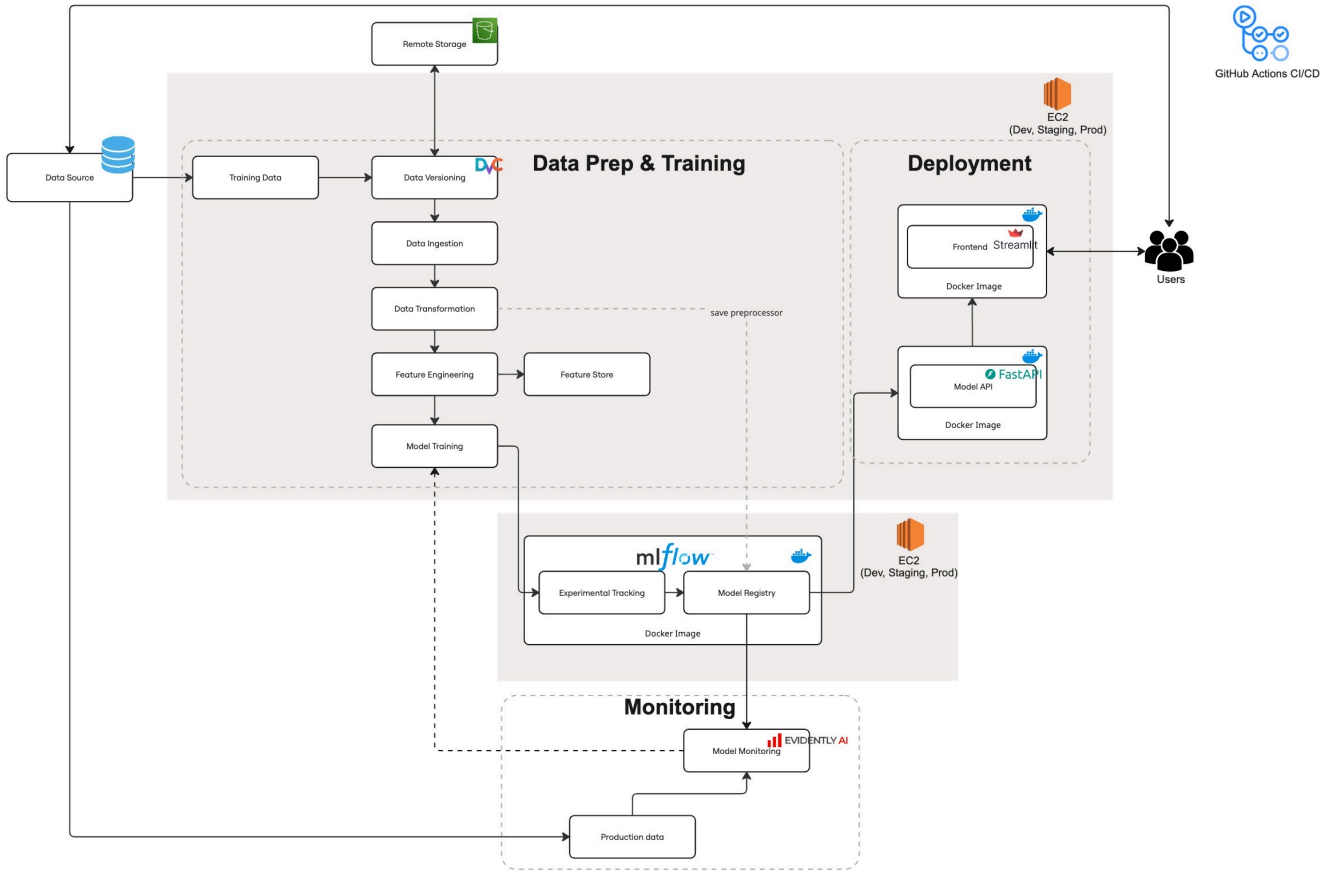
The Ultimate MLOps Stress Test

- Rapidly changing fraud patterns
- High stakes: real money, real consequences
- Need for fast, reliable model updates

step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
544	CASH_OUT	412668.39	C84071102	412668.39	0.0	C1576697216	1445047.47	1857715.86	1	0

Synthetic Fraud Dataset (Paysim)

2.3 My MLOps System Architecture



Roadmap

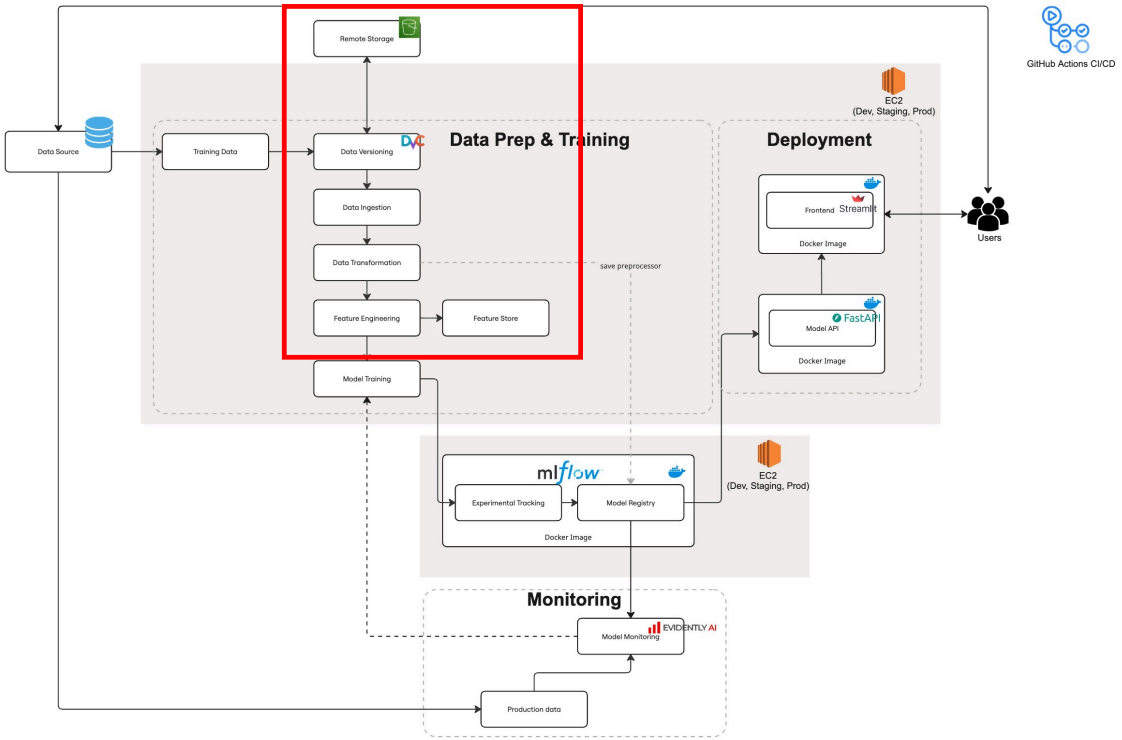
1. MLOps Motivation
2. System Overview & Use Case — Building the Foundation ✓
3. Pipeline Component — The Data & Model Pipeline
4. Infrastructure & Deployment— Scaling Across Environments

Demo

Roadmap

1. MLOps Motivation
2. System Overview & Use Case — Building the Foundation
3. Pipeline Component — The Data & Model Pipeline 
4. Infrastructure & Deployment— Scaling Across Environments

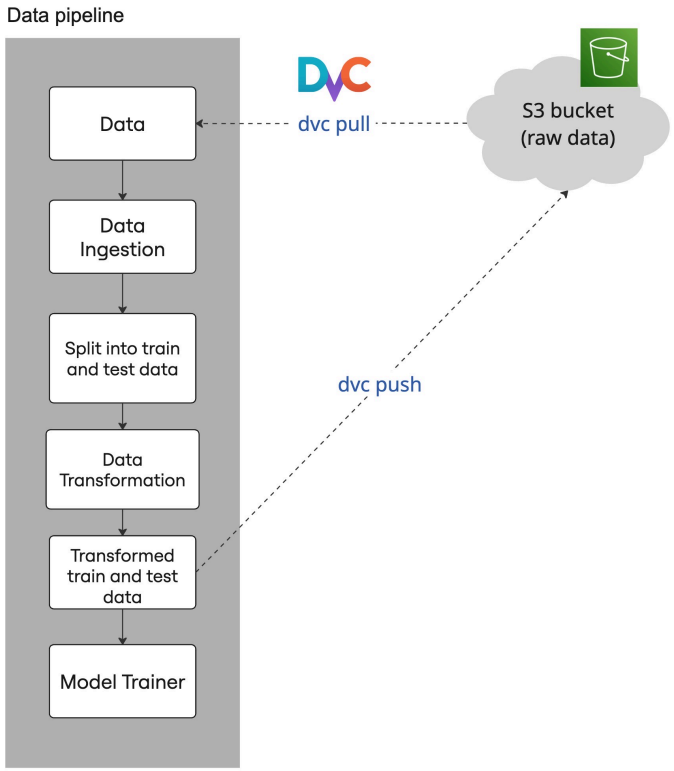
Demo



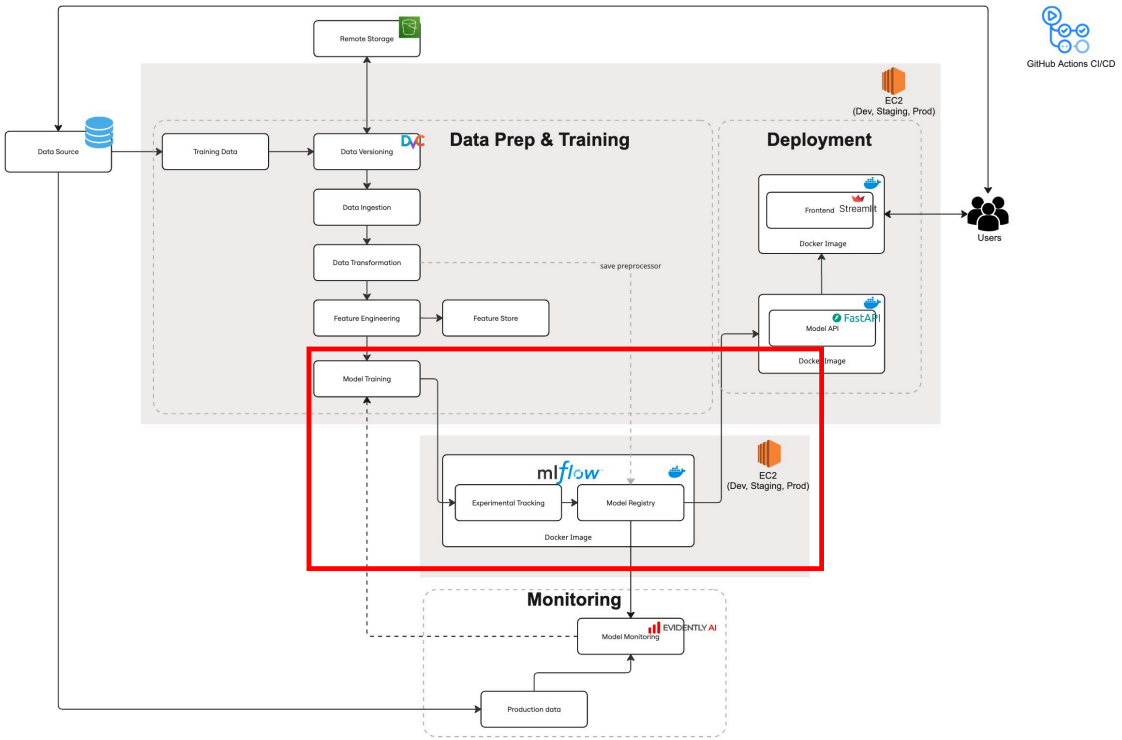
3.1 Data Preparation & Training

- Data management with DVC
- Data pipeline
- Model training and experimental tracking
- Model registry

3.1.1 Data Management & Versioning



1. Pull and push data to remote storage
2. Data versioning



3.1 Data Preparation & Training

- Data management with DVC
- Data pipeline
- Model training and experimental tracking
- Model registry

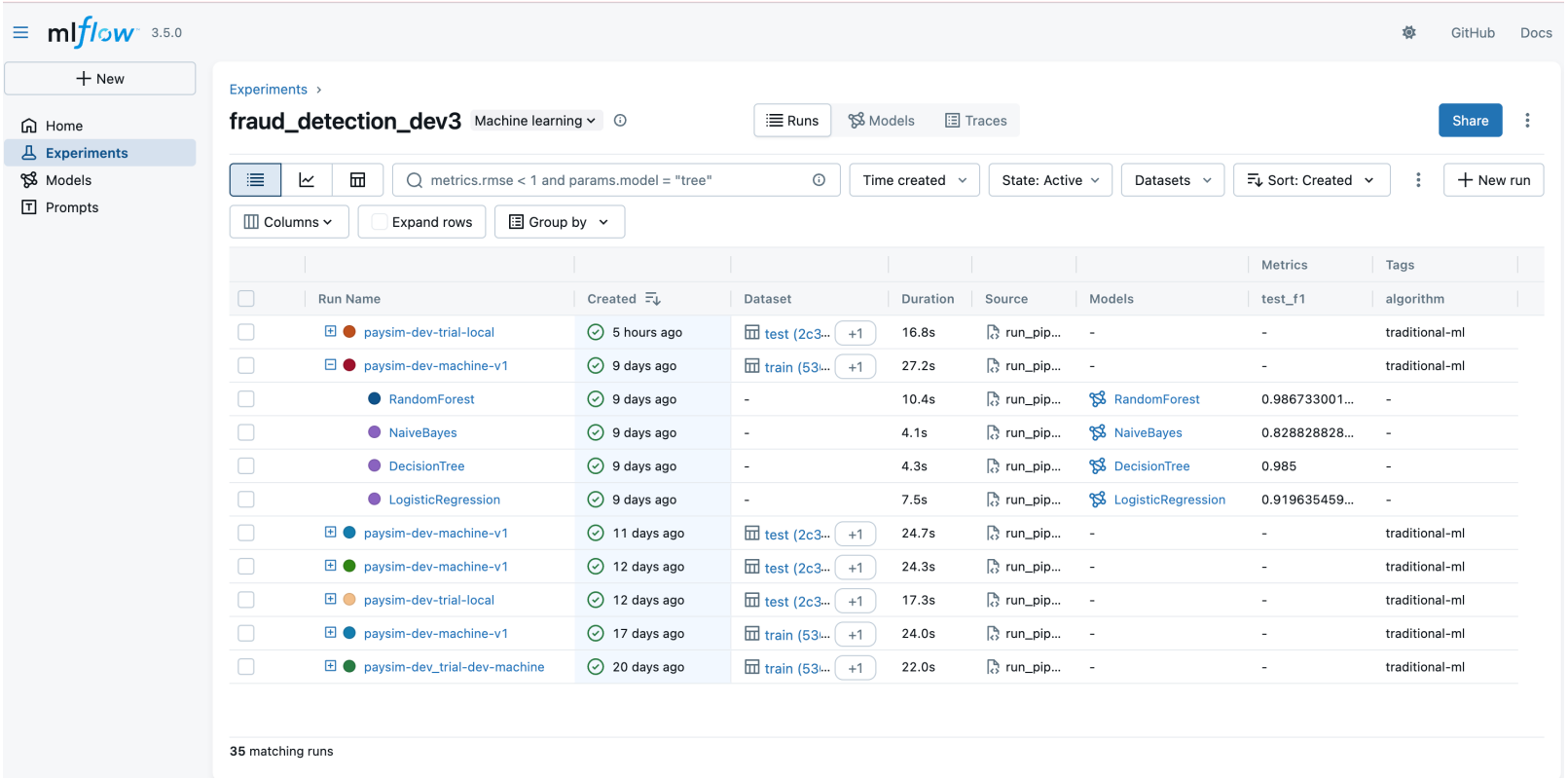
3.1.2 Model Training: Experimental Tracking

Problem: Manual, error-prone, not reproducible

	RESULTS								
							metrics		
	Run_id	Model	Standardisation	Datatype	Fe			Calinski-Harabasz (max)	Notes
baseline	77a25b4e89(	K-Means	Standard Scalar	numerical				1457	cluster_n = 7 is the best for silhouette, david, calinski, but not for inertia
best	bb4a55be19	K-Prototype	Standard Scalar	mixed				1141	
	0a41c5f612b	K-Prototype, optuna (composite)	Standard Scalar	mixed				1227.73	from Optuna_Kprotoype_Study_for_Composite_Metric. Composite score = 1227.73
	4bdebd639d	K-Prototype, optuna (best silhouette)	Standard Scalar	mixed				1130.8	from Optuna_Kprotoype_Study_for_each_metrics
	d8734d39e3(	K-Prototype, optuna (best Davies-Bouldin)	Standard Scalar	mixed				1010.4371	
	7245c4909c	K-Prototype, optuna (best Calinski-Harabasz)	Standard Scalar	mixed				1212.5887	
		DBSCAN	Standard Scalar	numerical				1.493	NA because device type is separated
		AgglomerativeClustering	Standard Scalar	mixed				2386.103	
								2153.153	
								1447.742	
		KMedoid	Standard Scalar	mixed				1163.834	
	OUTDATED!								
		BEST MODEL	EXPLANATION					CAVEATS	
	1	K-Prototype	Good Silhouette Score (0.555) -> In						
			Lowest Davies-Bouldin Score (0.528						
			Highest Calinski-Harabasz Score (7					cluster variance → well-separated clusters.	
			Works on Mixed Data						
	2	AgglomerativeClustering (cosine)	Highest silhouette score (0.763) → s					k=2 doesn't make sense (take higher k, sacrificing some accuracy)	
			Very good Calinski-Harabasz (2386.1) → clusters are well-separated.						
			Davies-Bouldin isn't the lowest, but still acceptable (0.974).						
			Clearly defined optimal_k = 2, , and cosine distance seems best for this structure.						
			Works on Mixed Data						

3.1.2 Model Training: Experimental Tracking

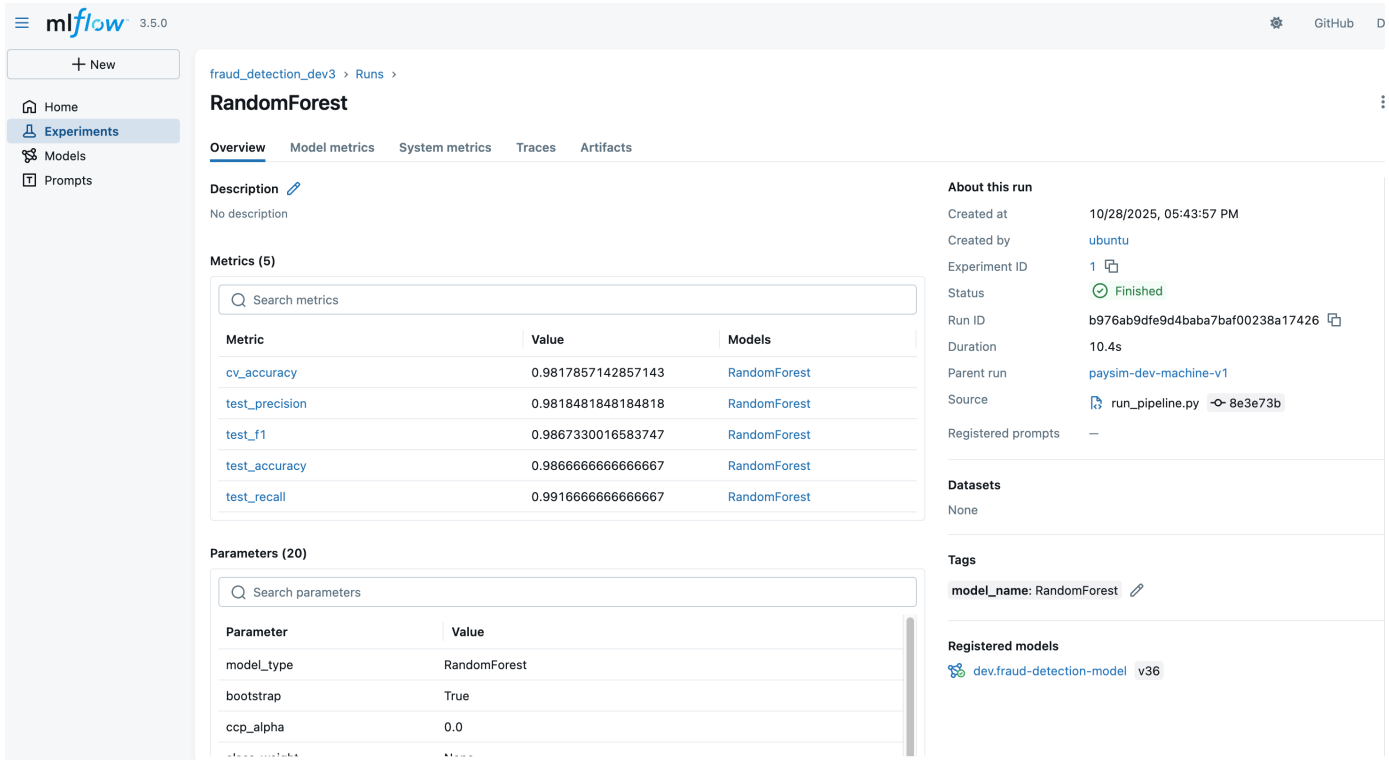
Solution: MLflow



Each experimental run trains 4 ML models. Logs metrics, preprocessor and model artifacts, etc.

3.1.2 Model Training: Experimental Tracking

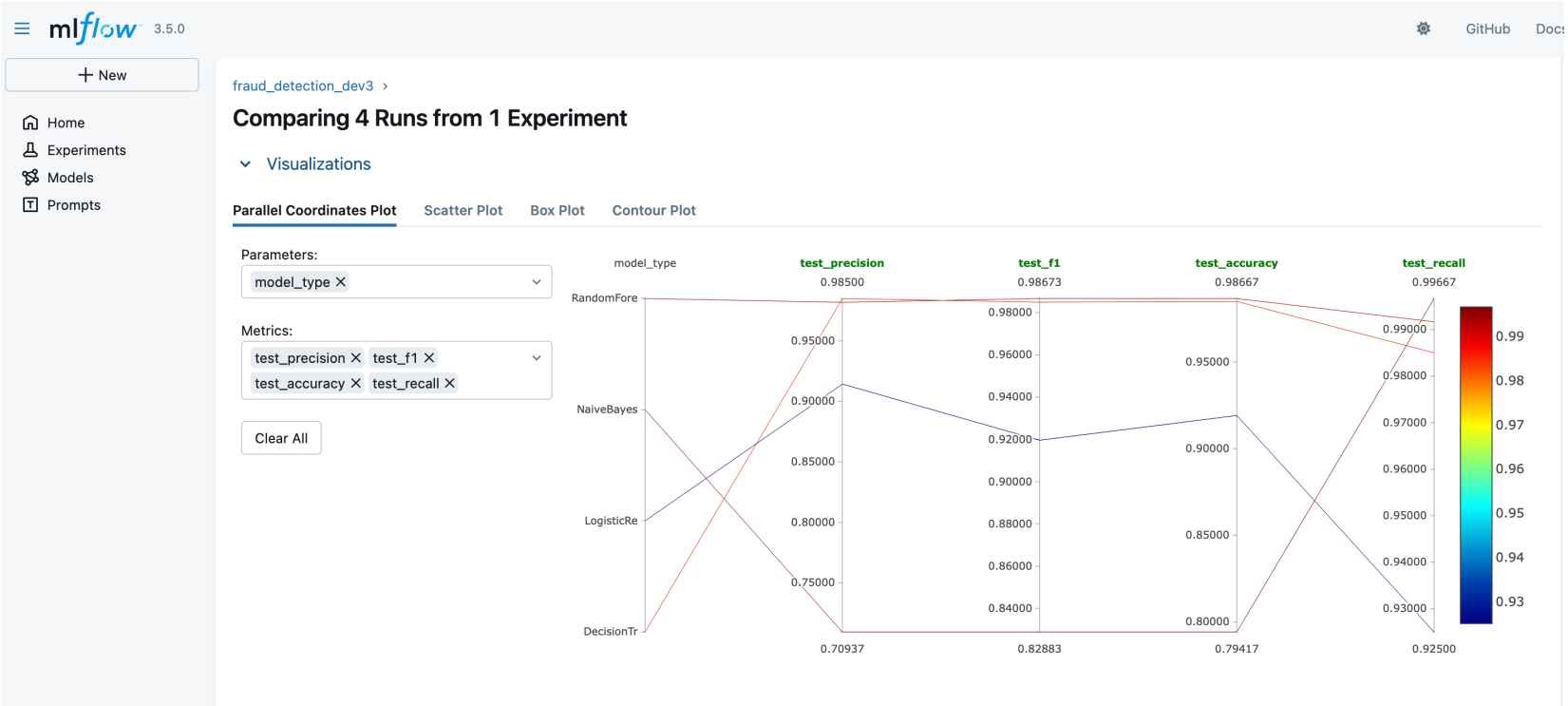
Solution: MLflow



View a specific model – see metrics and parameters

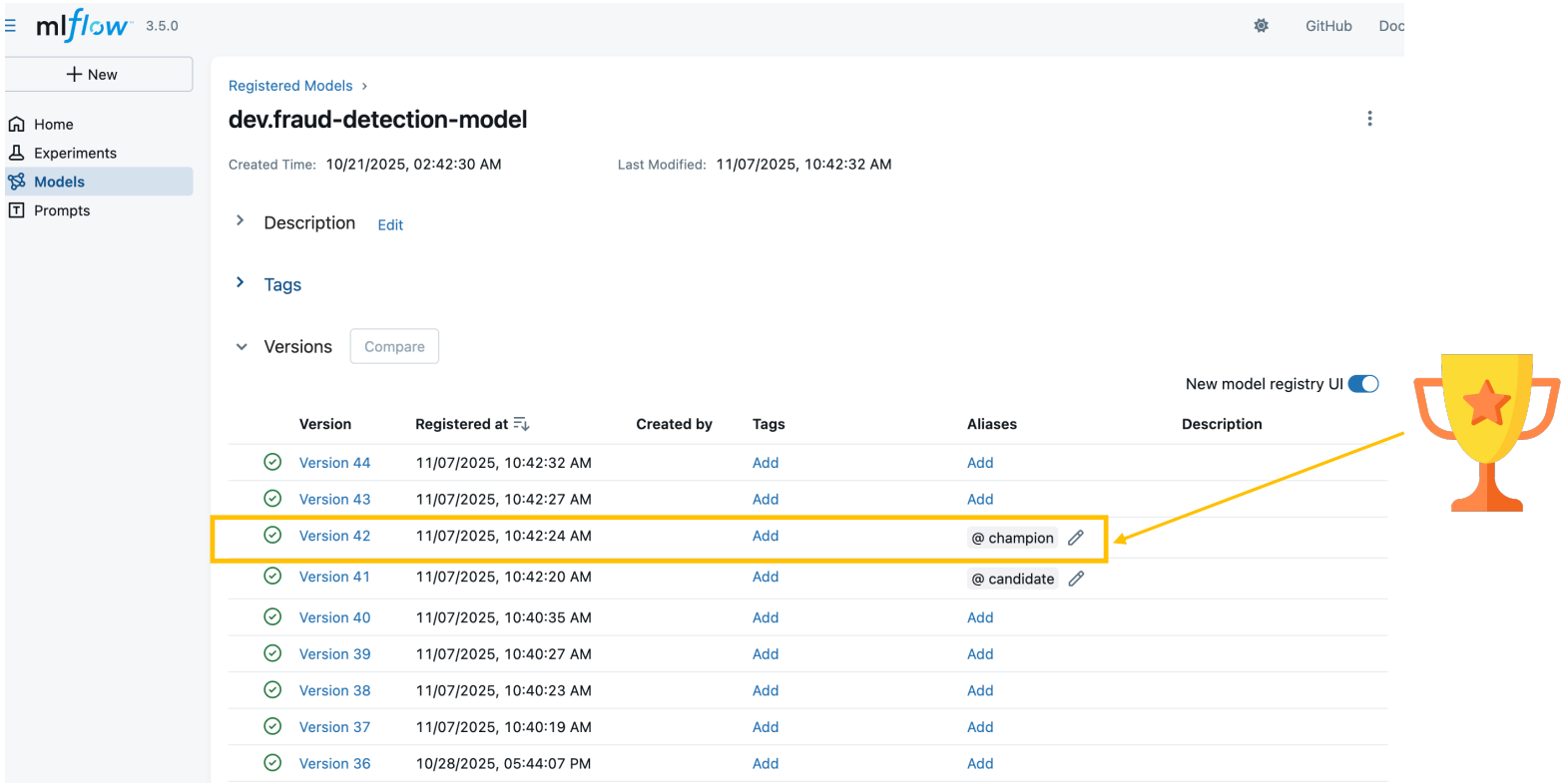
3.1.2 Model Training: Experimental Tracking

Solution: MLflow

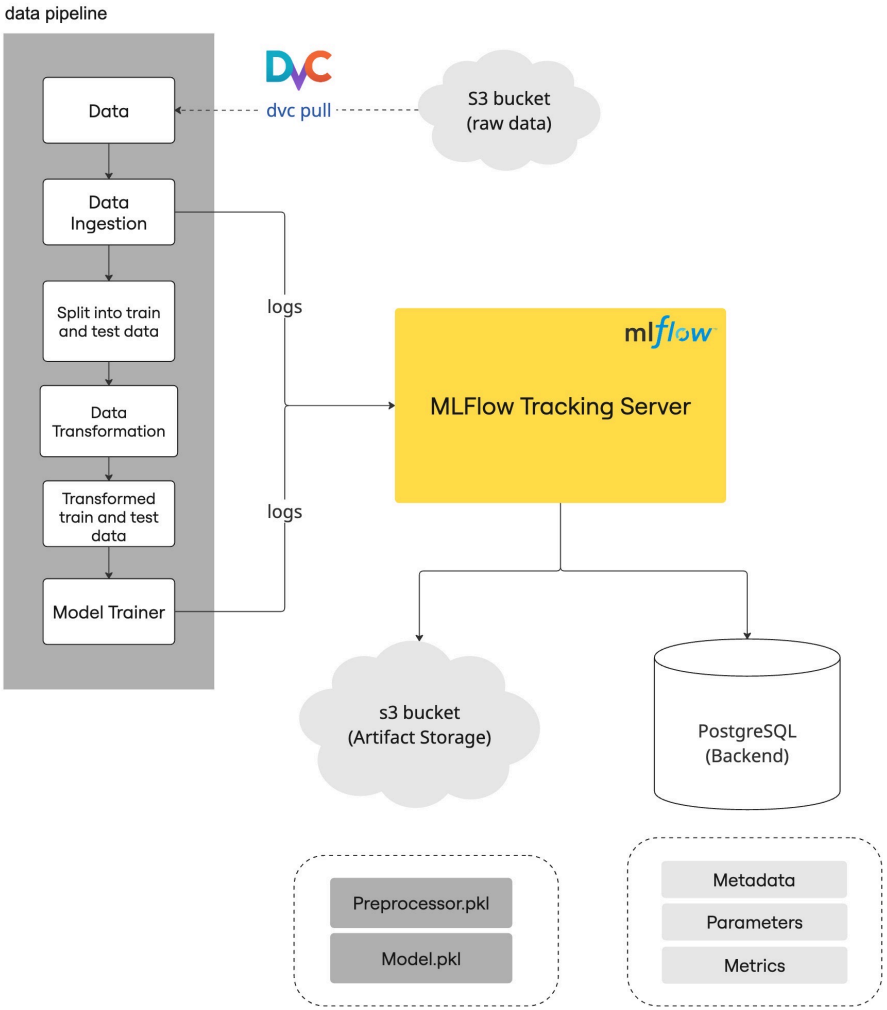


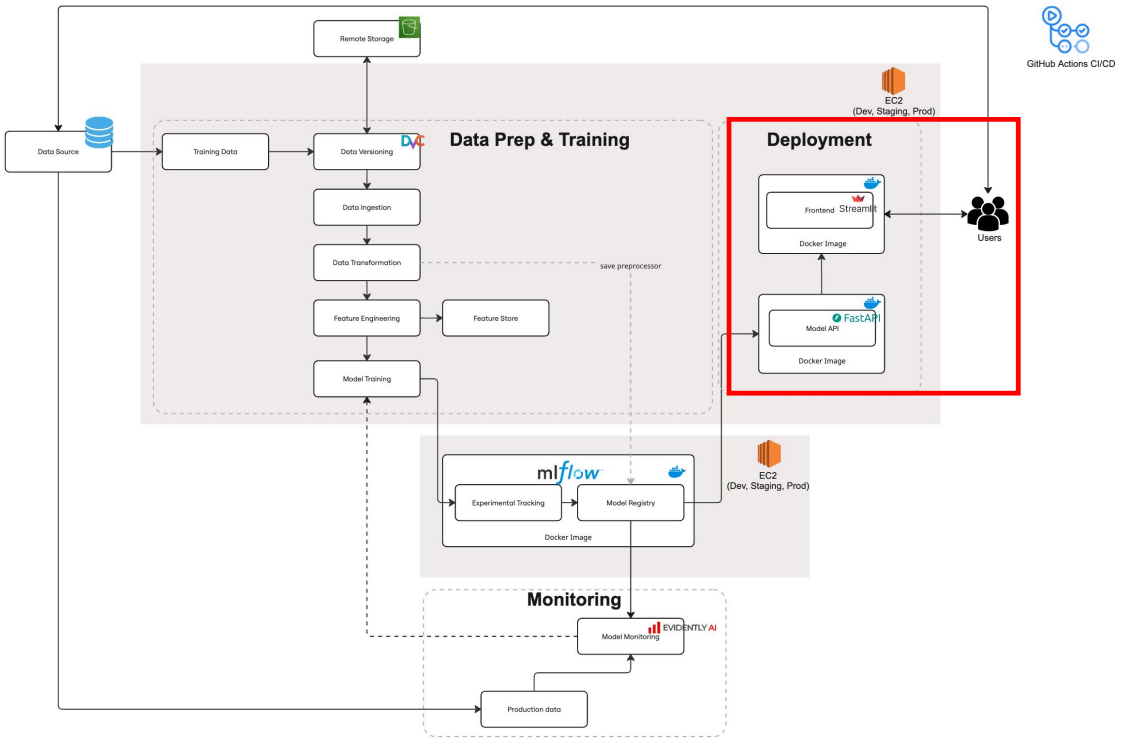
Easily compare metrics of trained models

3.1.3 Model Registry: Choose the best model



The best model is labelled as "champion"

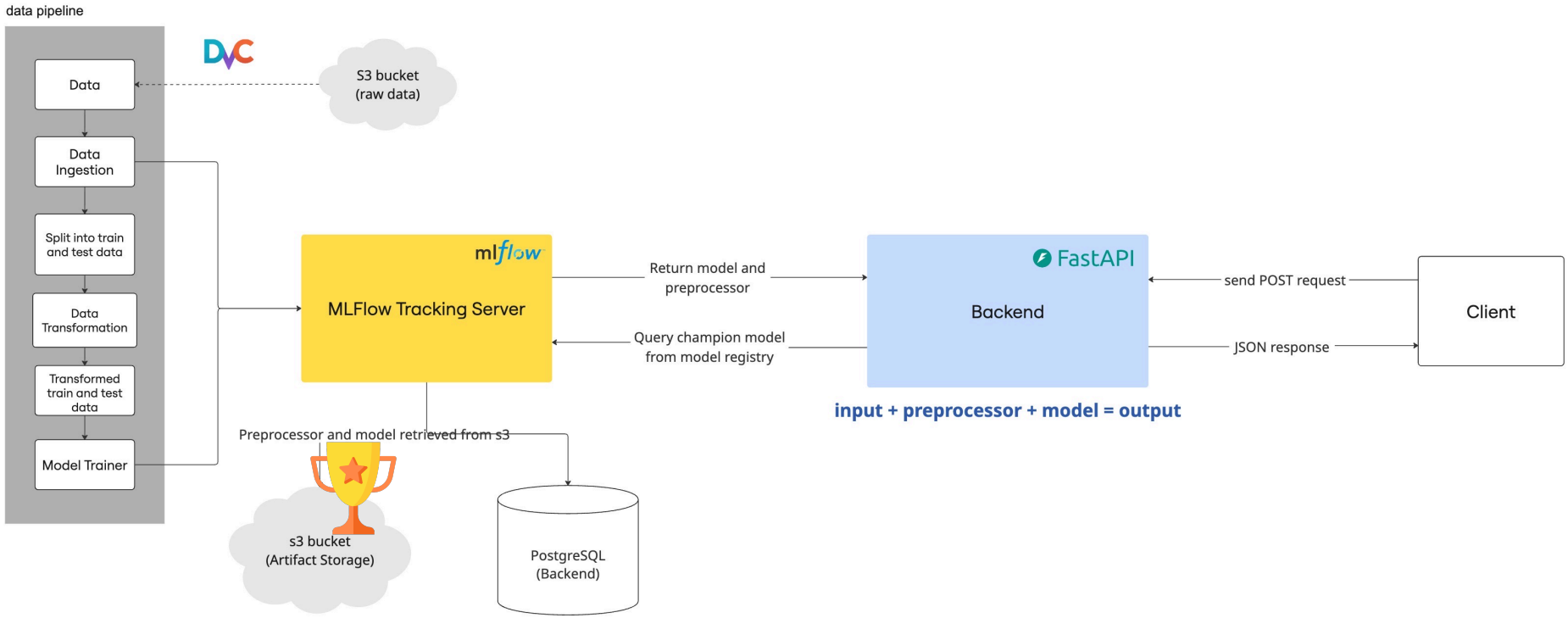

data pipeline
Data
dvc pull
S3 bucket
(raw data)
Data
Ingestion
Split into train
and test data
Data
Transformation
Transformed
train and test
data
Model Trainer
logs
logs
mlflow
MLFlow Tracking Server
s3 bucket
(Artifact Storage)
PostgreSQL
(Backend)
Preprocessor.pkl
Model.pkl
Metadata
Parameters
Metrics



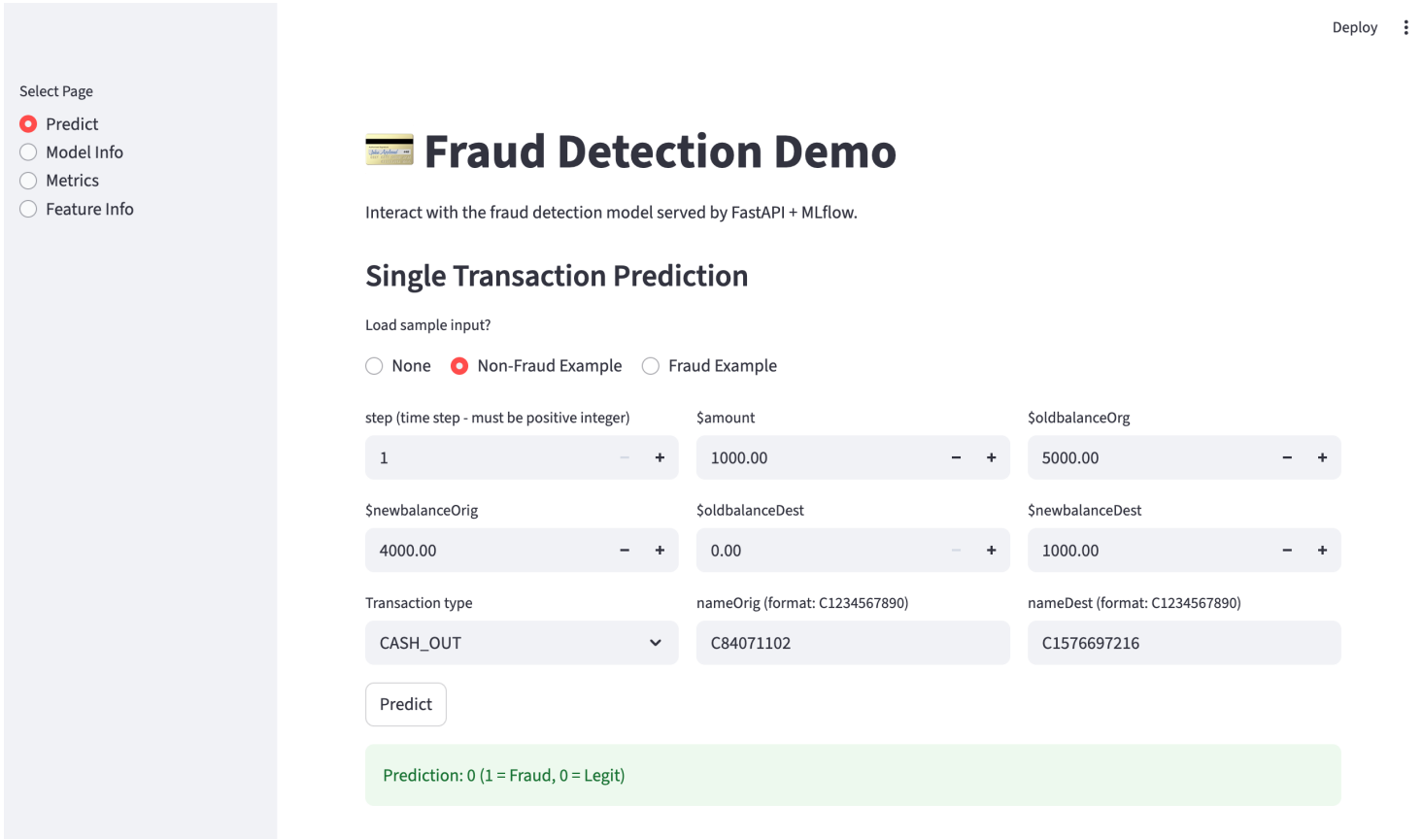
3.2 Deployment

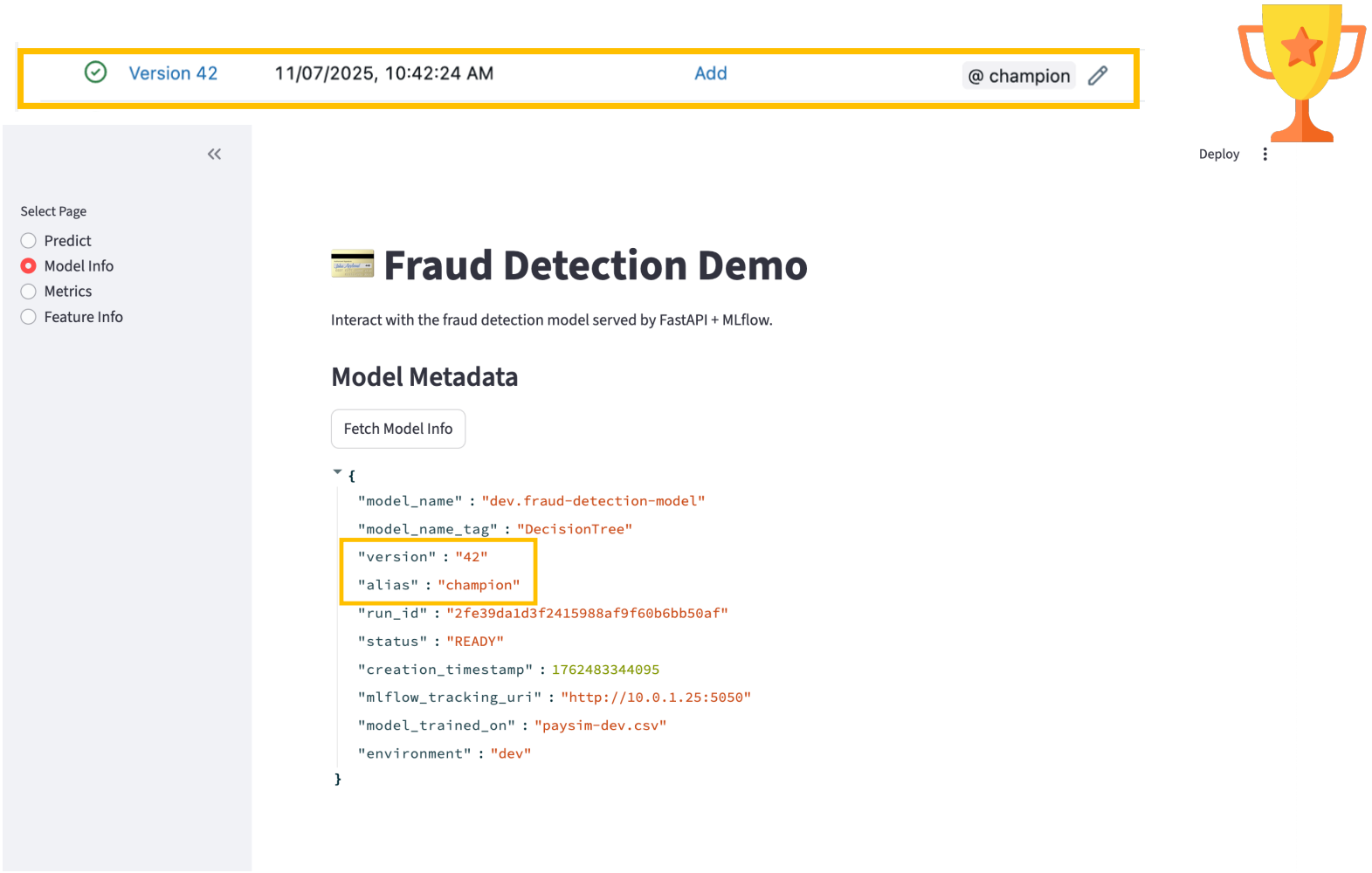
- FastAPI backend
- Streamlit frontend

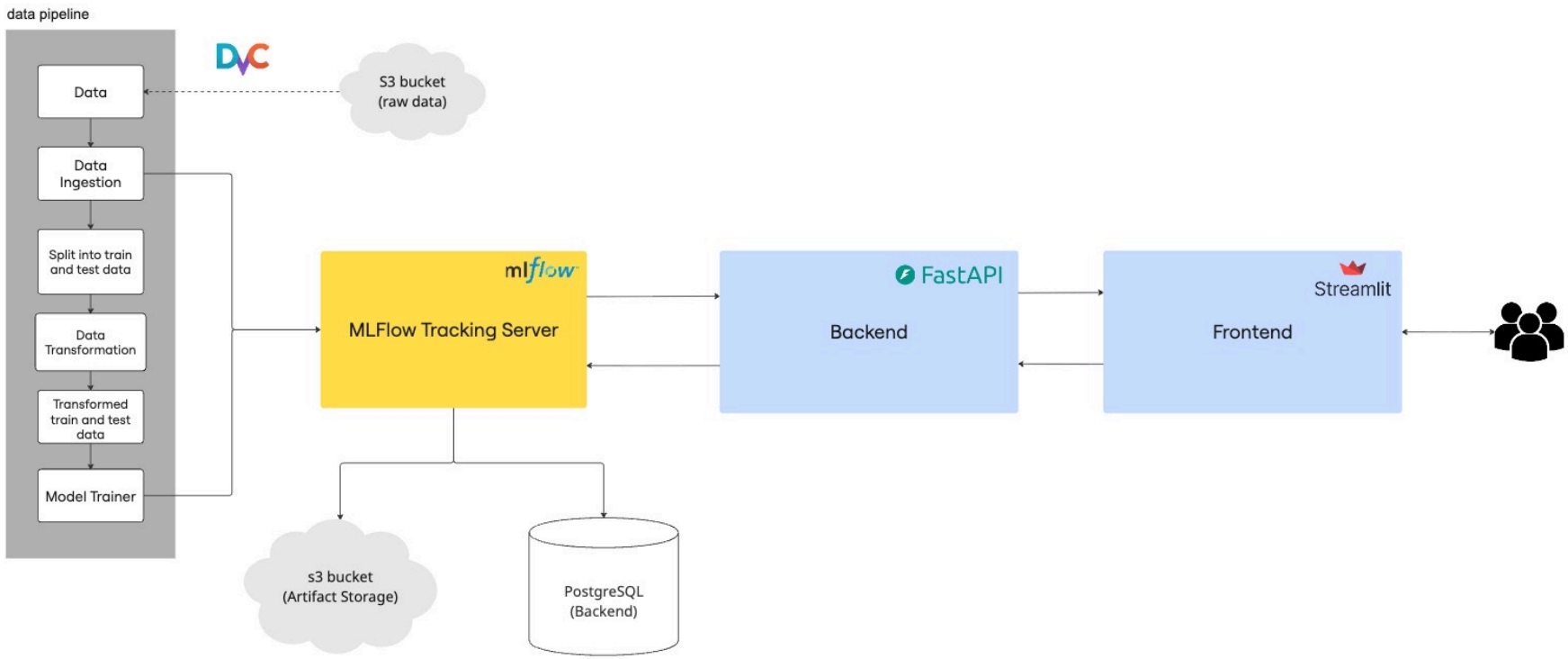
3.2.1 Backend: FastAPI



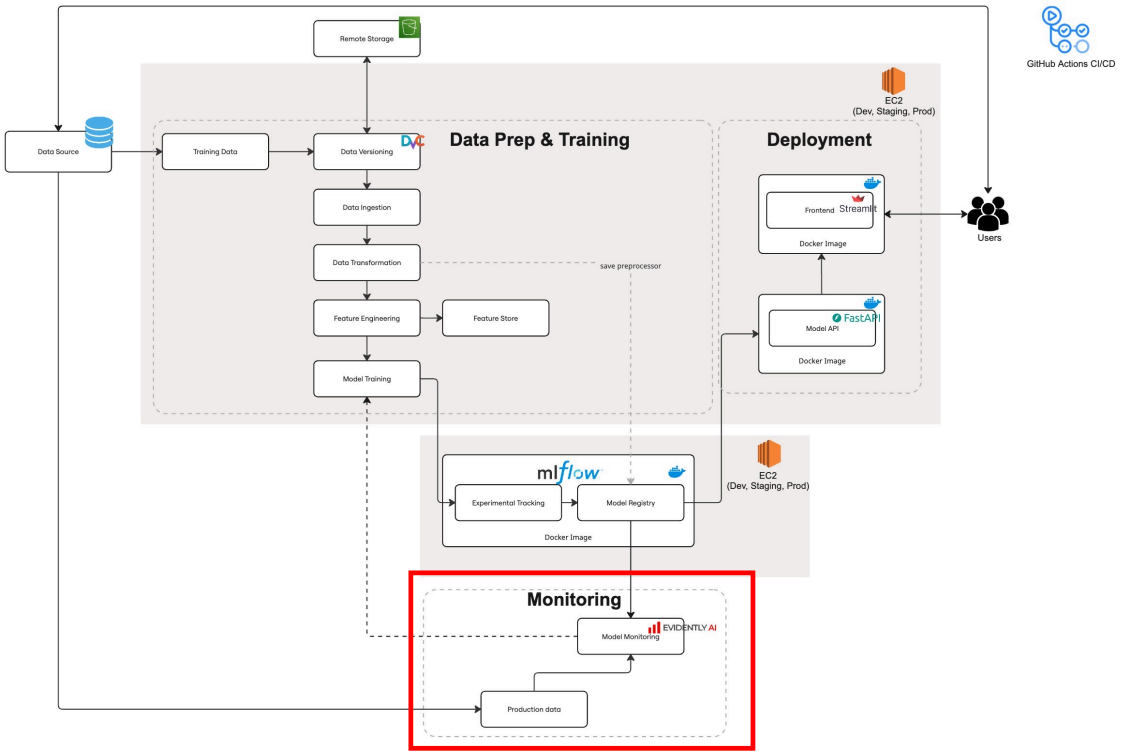
3.2.2 Frontend: Streamlit




Version 42
11/07/2025, 10:42:24 AM
Add
@ champion
Deploy
Select Page
Predict
Model Info
Metrics
Feature Info
Fraud Detection Demo
Interact with the fraud detection model served by FastAPI + MLflow.
Model Metadata
Fetch Model Info
{
"model_name" : "dev.fraud-detection-model"
"model_name_tag" : "DecisionTree"
"version" : "42"
"alias" : "champion"
"run_id" : "2fe39da1d3f2415988af9f60b6bb50af"
"status" : "READY"
"creation_timestamp" : 1762483344095
"mlflow_tracking_uri" : "http://10.0.1.25:5050"
"model_trained_on" : "paysim-dev.csv"
"environment" : "dev"
}


data pipeline
DVC
Data
S3 bucket
(raw data)
Data
Ingestion
Split into train
and test data
Data
Transformation
Transformed
train and test
data
Model Trainer
mlflow
MLFlow Tracking Server
FastAPI
Backend
Streamlit
Frontend
s3 bucket
(Artifact Storage)
PostgreSQL
(Backend)

3.3 Monitoring



- Model monitoring and drift detection with Evidently AI

3.3 Model monitoring and drift detection

- Track model performance in production
- Detect data drift and concept drift
- Automated alerts for performance drops, and model retraining
- Use Evidently AI for monitoring and visualisation

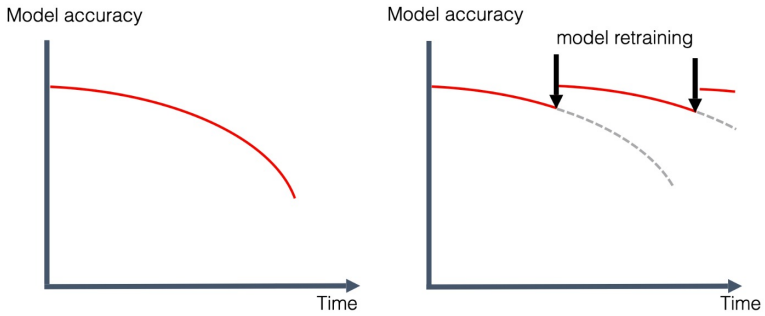


Image source: Evidently AI docs

Roadmap

1. MLOps Motivation
2. System Overview & Use Case — Building the Foundation
3. Pipeline Component — The Data & Model Pipeline ✓
4. Infrastructure & Deployment— Scaling Across Environments

Demo

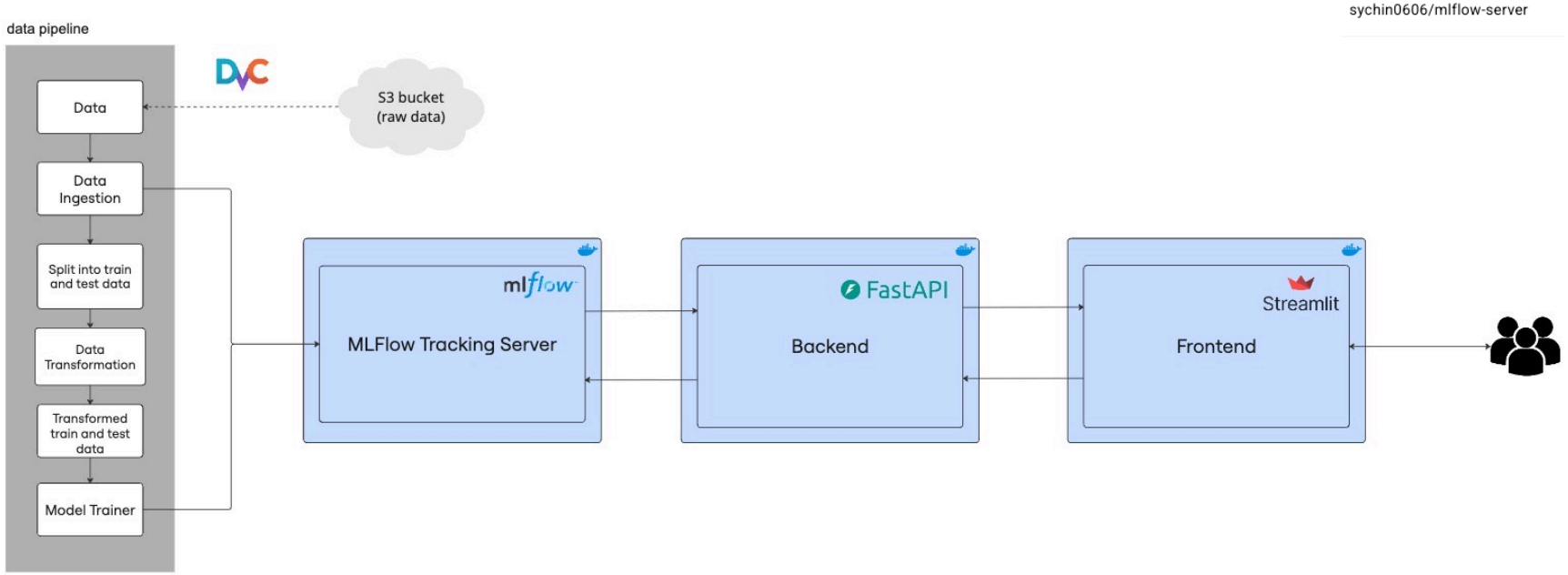
Roadmap

1. MLOps Motivation
2. System Overview & Use Case — Building the Foundation
3. Pipeline Component — The Data & Model Pipeline
4. Infrastructure & Deployment— Scaling Across Environments

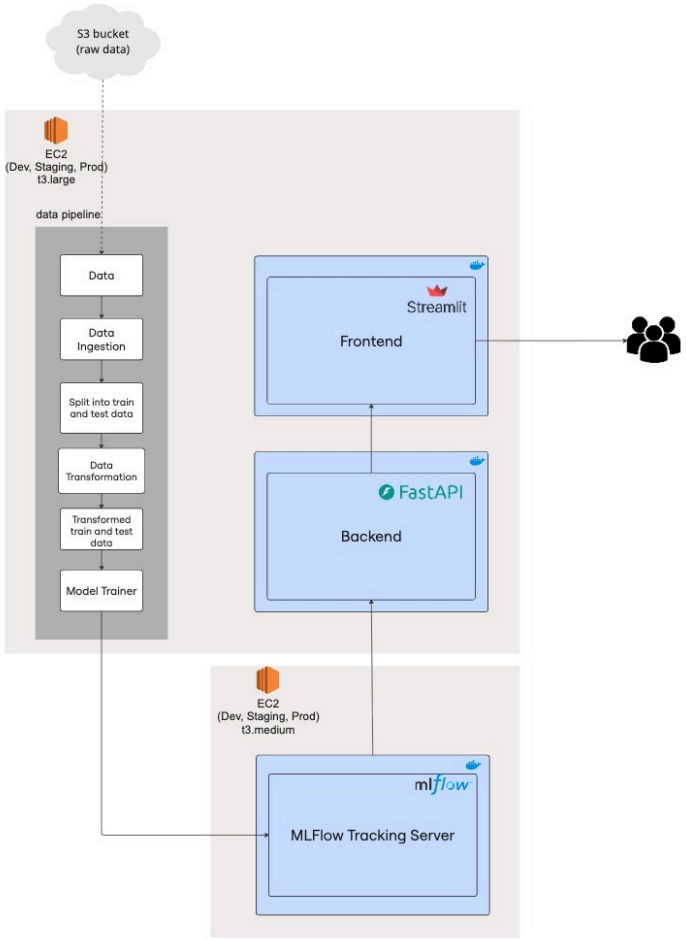
Demo

4.1 Docker containerisation

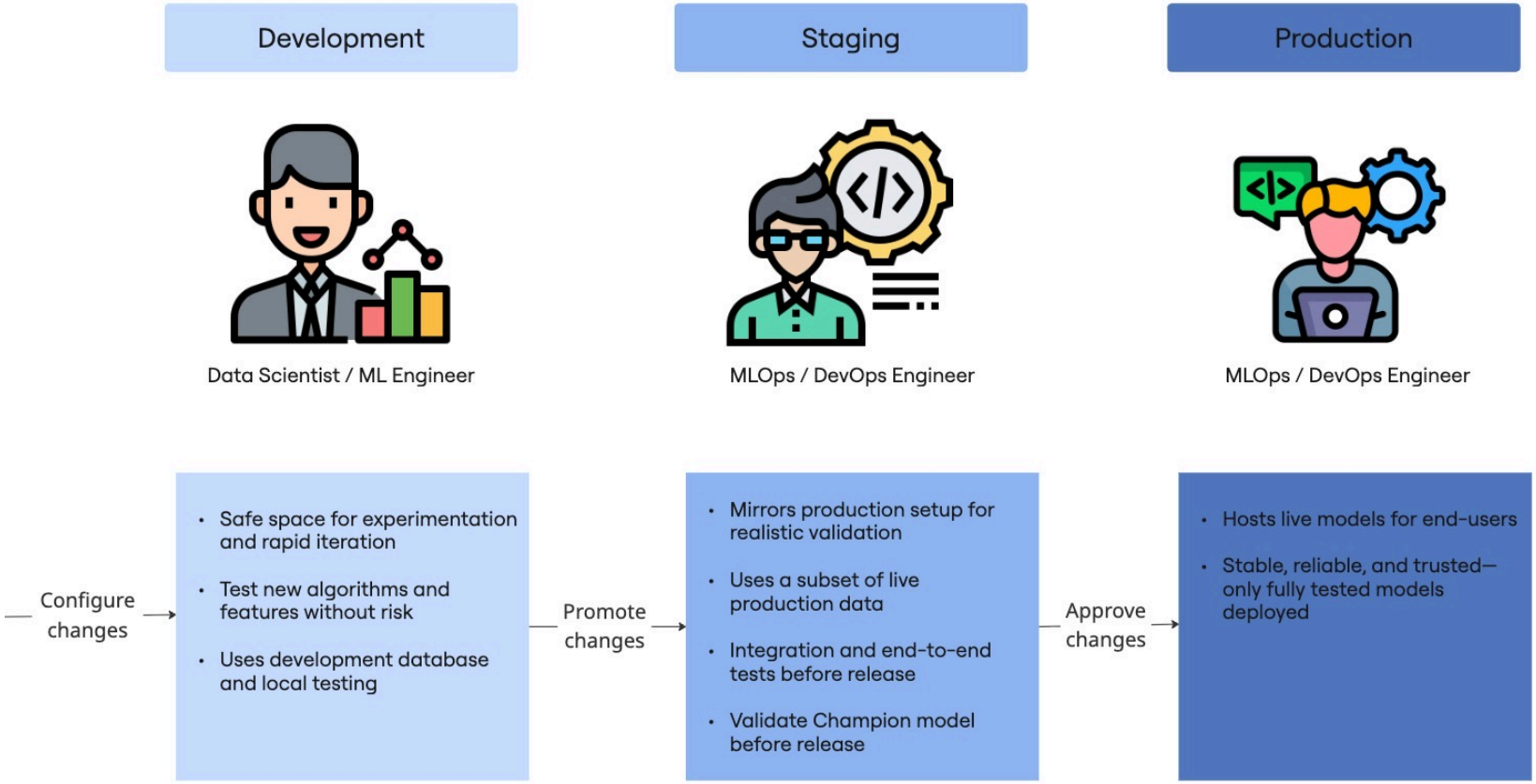
Name
sychin0606/frontend
sychin0606/backend
sychin0606/mlflow-server

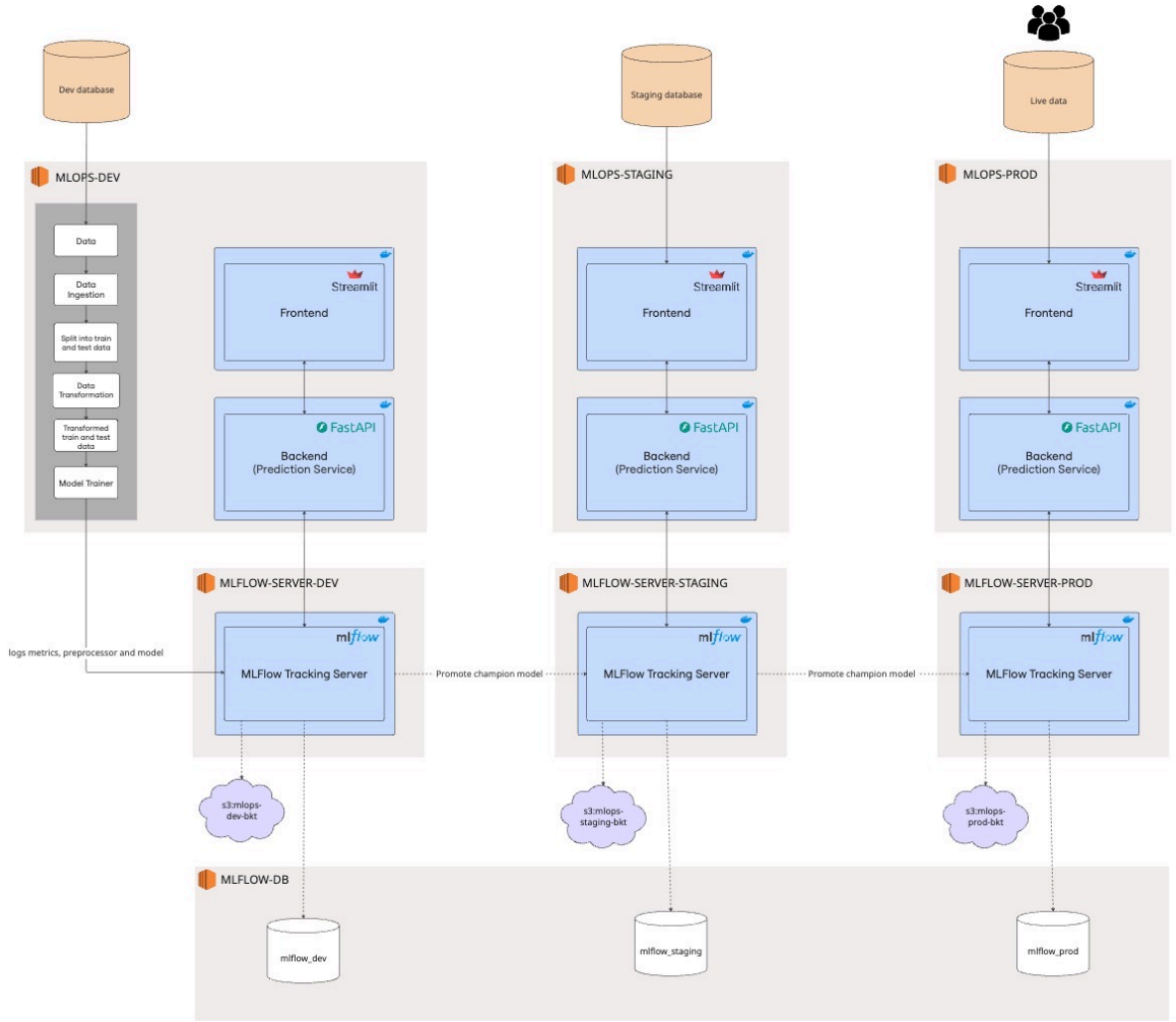


4.2 Virtual Machines: AWS EC2

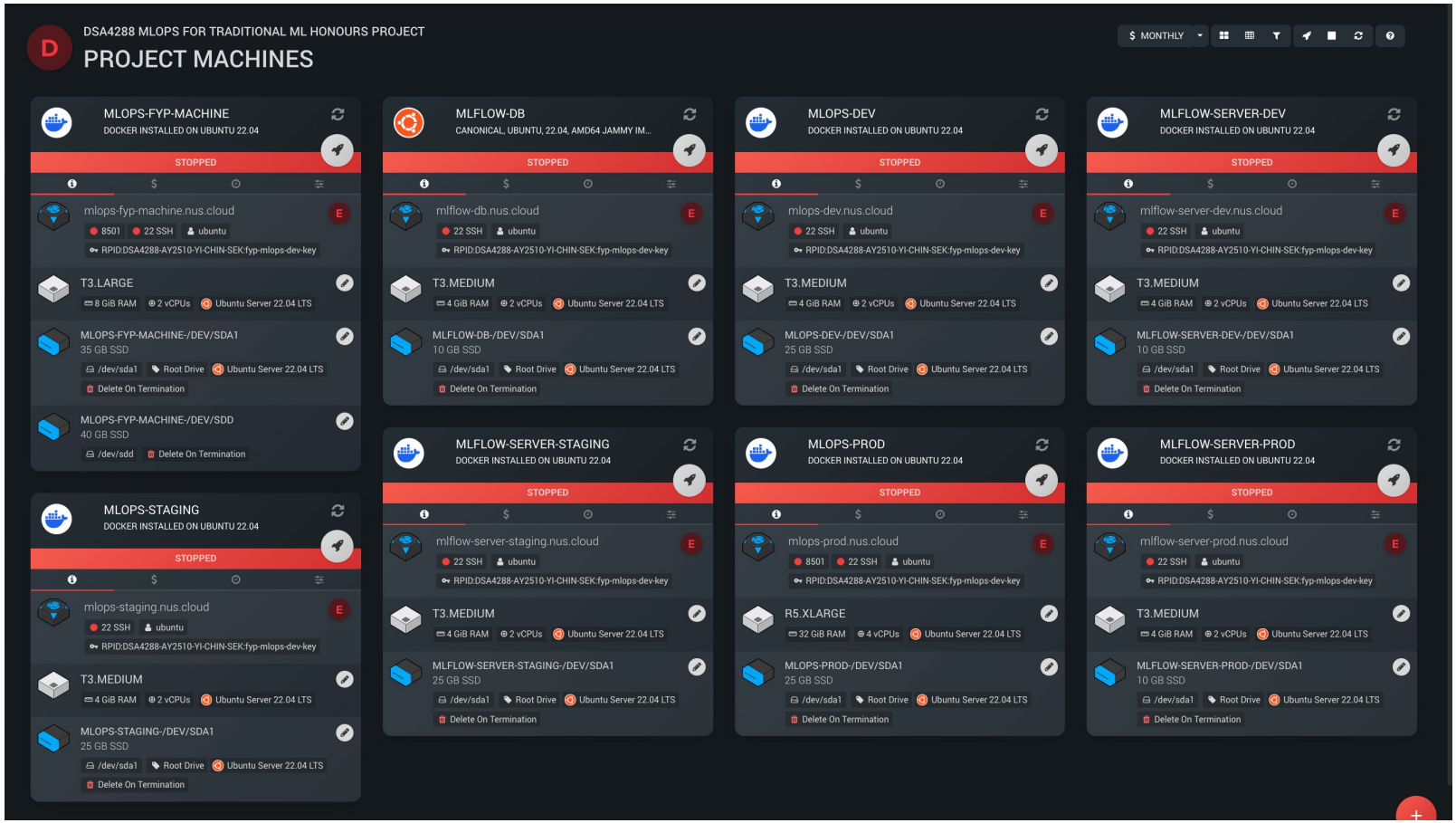


4.3 Multi-Environment: Dev, Staging, Production




Dev database
Staging database
Live data
MLOPS-DEV
Data
Data Ingestion
Split into train and test data
Data Transformation
Transformed train and test data
Model Trainer
Streamlit
Frontend
FastAPI
Backend
(Prediction Service)
MLOPS-STAGING
MLOPS-PROD
MLFLOW-SERVER-DEV
MLFLOW-SERVER-STAGING
MLFLOW-SERVER-PROD
mlflow
MLFlow Tracking Server
logs metrics, preprocessor and model
Promote champion model
Promote champion model
s3:mlops-dev-bkt
s3:mlops-staging-bkt
s3:mlops-prod-bkt
MLFLOW-DB
mlflow_dev
mlflow_staging
mlflow_prod

Multi-Environment: Dev, Staging, Production



4.4 How we separate Dev, Staging, Production

Different configuration usings different:

- EC2 instances
- Backend databases
- S3 buckets
- .env files
- Docker compose files

Component / Purpose	Dev	Staging	Prod
EC2 Instance Name	MLFLOW-SERVER-DEV	MLFLOW-SERVER-STAGING	MLFLOW-SERVER-PROD
PostgreSQL Database	mlflow_dev	mlflow_staging	mlflow_prod
MLflow Artefacts S3	s3://mlops-dev-bkt/	s3://mlops-stag-bkt/	s3://mlops-prod-bkt/
Docker Compose File	docker-compose.mlflow-server-dev.yml	docker-compose.mlflow-server-staging.yml	docker-compose.mlflow-server-prod.yml
Environment File	.env.mlflow_dev	.env.mlflow_stag	.env.mlflow_prod

Table 4.2: MLflow Server Configuration Across Environments

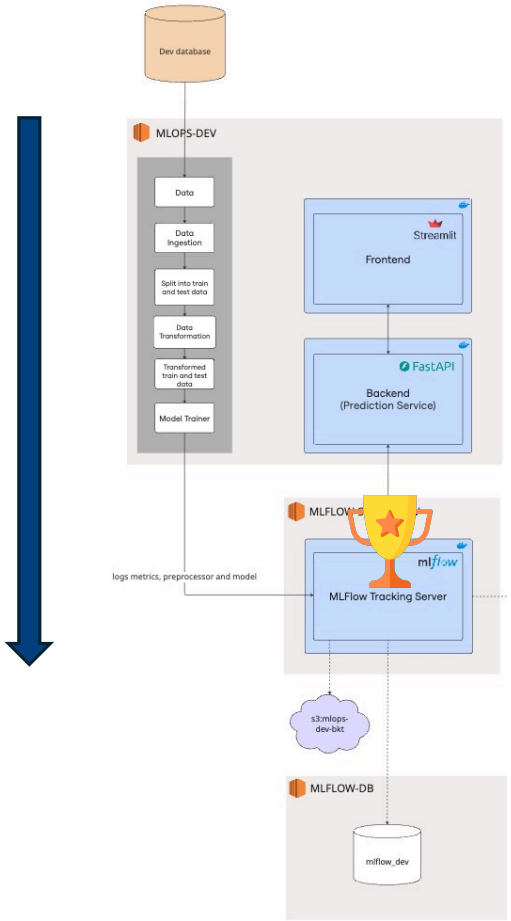
Component / Purpose	Dev	Staging	Prod
EC2 Instance Name	MLOPS-DEV	MLOPS-STAGING	MLOPS-PROD
Raw Data S3 Bucket	s3://mlops-dev-raw/	s3://mlops-stag-raw/	s3://mlops-prod-raw/
Docker Compose File	docker-compose.dev.yml	docker-compose.staging.yml	docker-compose.prod.yml
Environment File	.env.dev_machine	.env.stag_machine	.env.prod_machine
Download data	make download-data-dev	make download-data-staging	make download-data-prod
Spin up services	make up-dev	make up-staging	make up-prod
Model promotion	make promote-model-dev	make promote-model-staging	-
Testing	make test-dev	make test-staging	make test-prod

Table 4.3: Backend + Frontend Configuration Across Environments

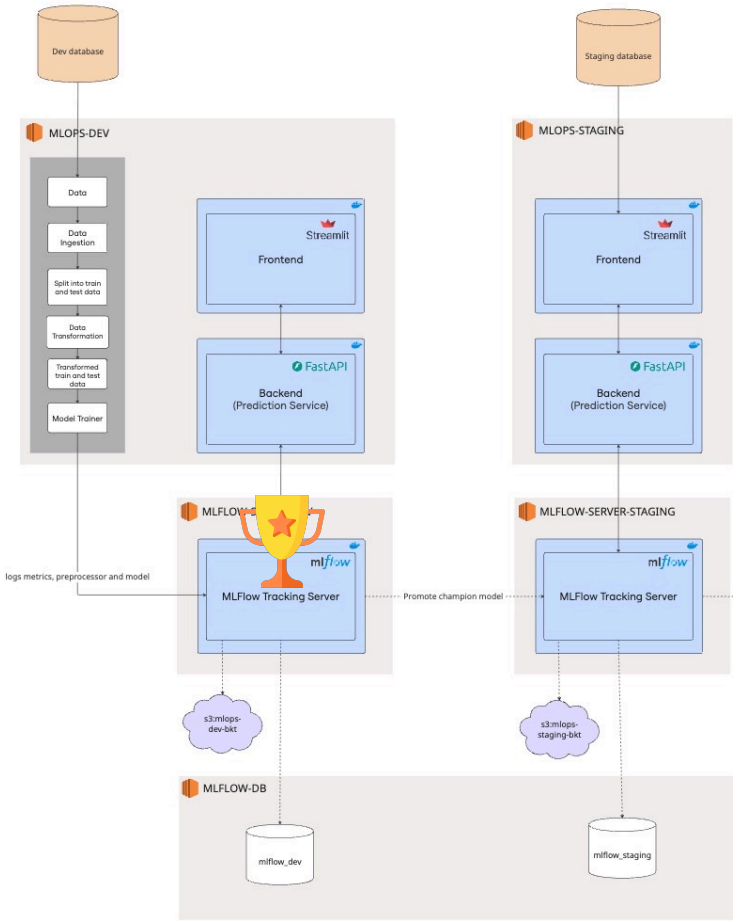
How do our Champion Model move through the 3 stages to reach end users?

Model Promotion Workflow

4.5 Model Promotion Workflow Dev → Staging → Production

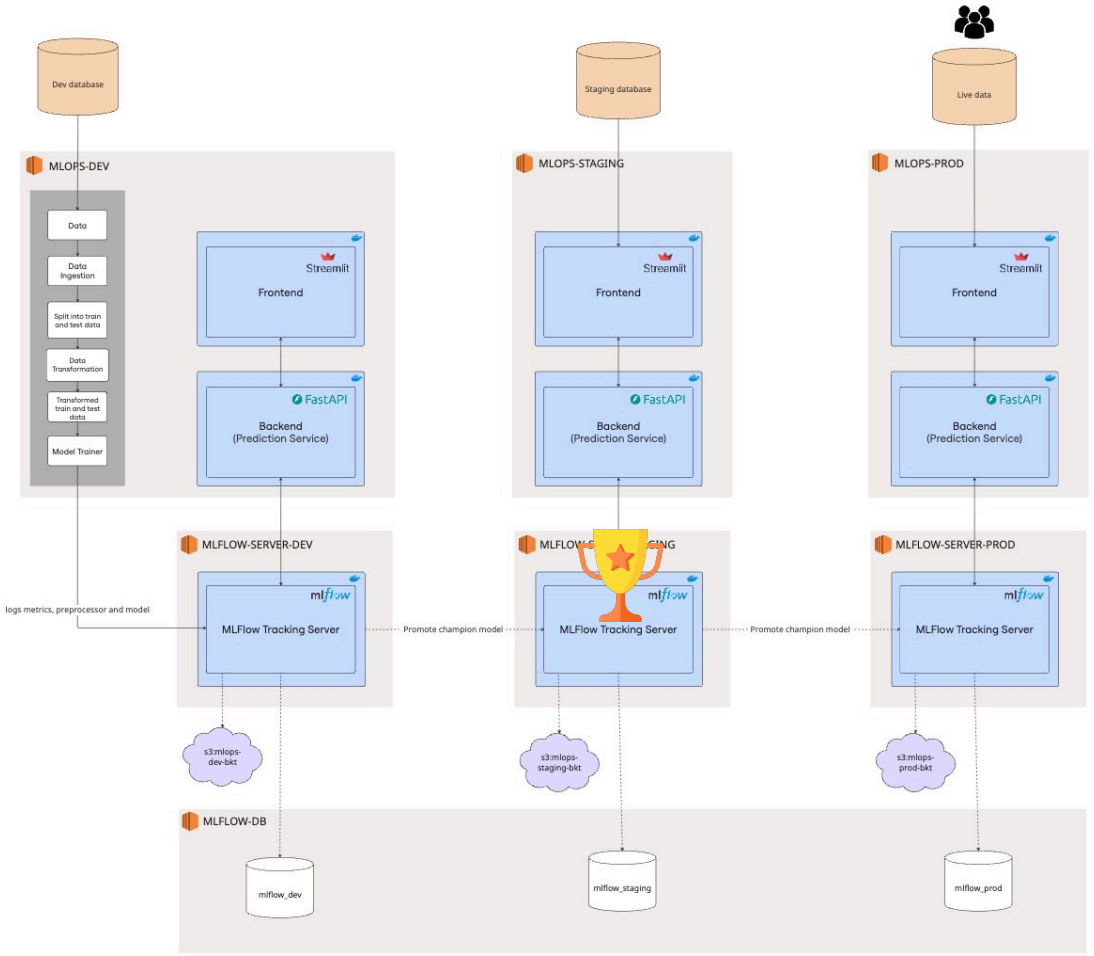


4.5 Model Promotion Workflow Dev → Staging → Production



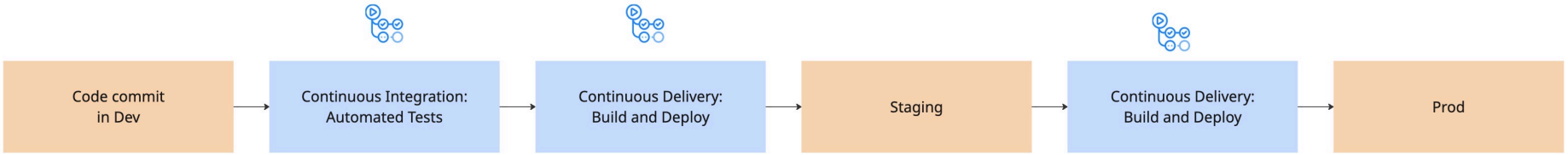

```
promote-model-dev:
    $(MAKE) download-champion-dev
    $(MAKE) upload-champion-to-staging
    AWS_PROFILE=dev-bkt AWS_DEFAULT_PROFILE=dev-bkt
```

4.5 Model Promotion Workflow Dev → Staging → Production




```
promote-model-staging:
    $(MAKE) download-champion-staging
    $(MAKE) upload-champion-to-prod
    AWS_PROFILE=stag-bkt AWS_DEFAULT_PROFILE=stag-bkt
```

How do we automate the journey from code and models to production—while catching errors before they reach users?

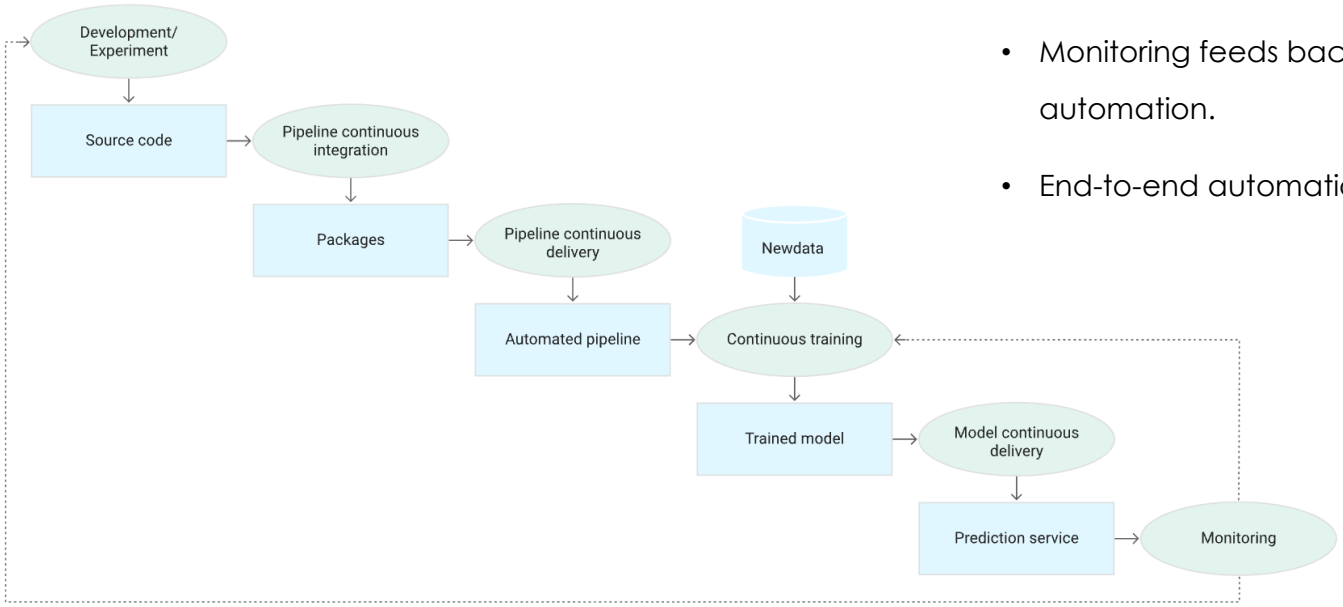


4.6 CI/CD

Automating Integration, Deployment, and Testing

- **CI (Continuous Integration):** Automatically tests and validates every code/model change.
- **CD (Continuous Deployment):** Packages and deploys applications/models to staging and production.
- Ensures reliability, reproducibility, and fast delivery across all environments.
- Reduces manual errors and mirrors industry best practices.

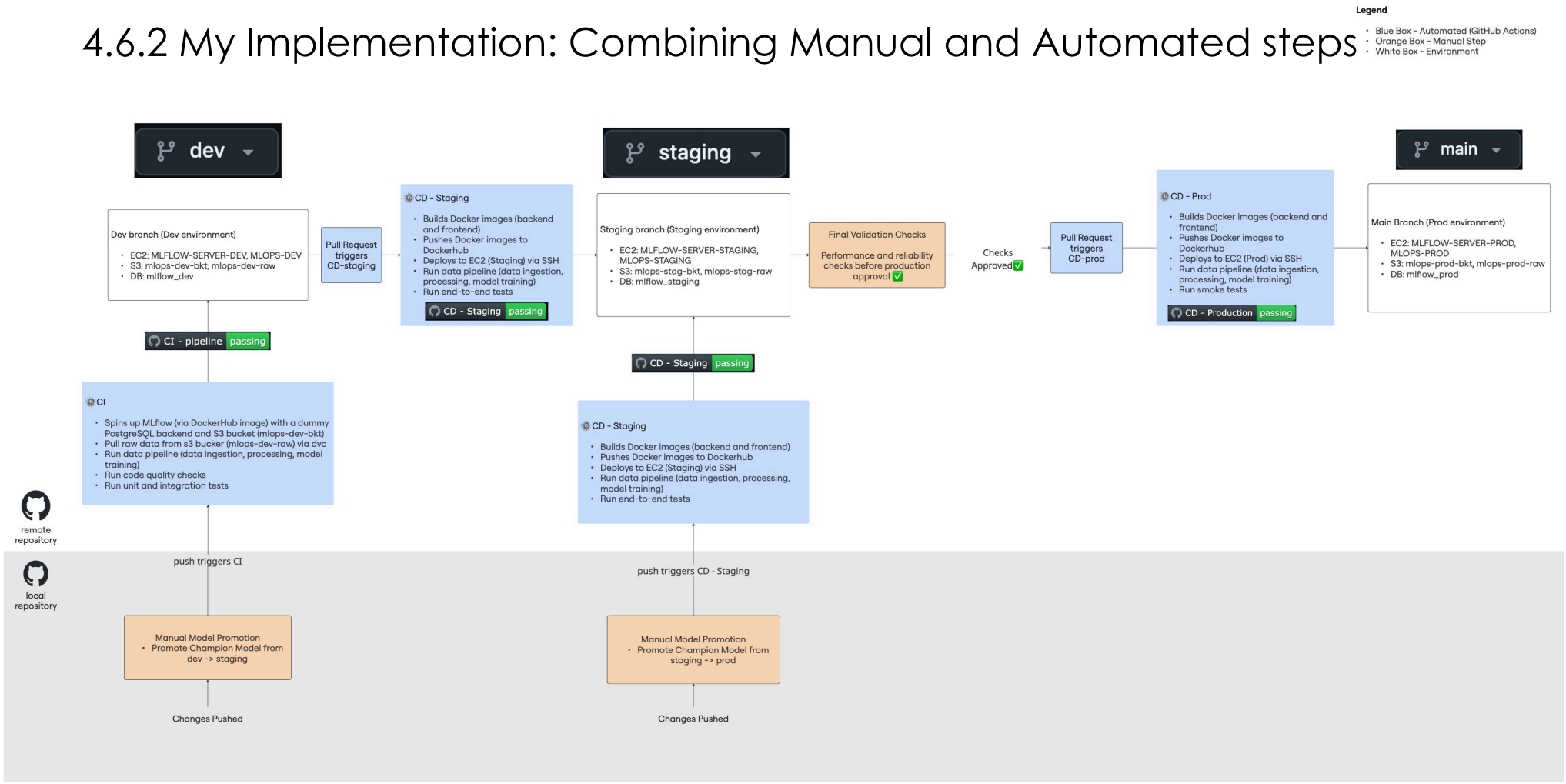
4.6.1 Gold Standard: Fully Automated ML CI/CD (Google Cloud)



- Continuous integration and delivery for both code and models.
- Automated retraining triggered by new data.
- Monitoring feeds back to retraining pipeline for true ML automation.
- End-to-end automation: minimal manual intervention.

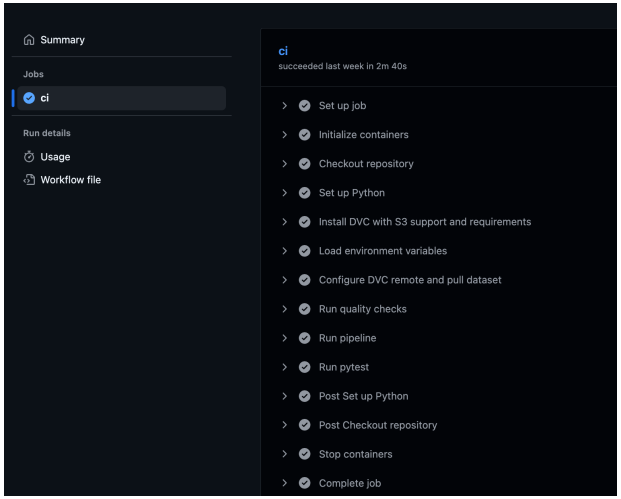
Source: Google cloud

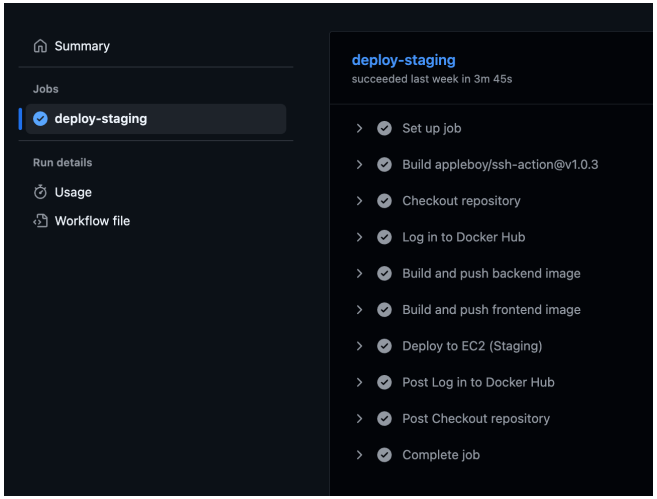
4.6.2 My Implementation: Combining Manual and Automated steps

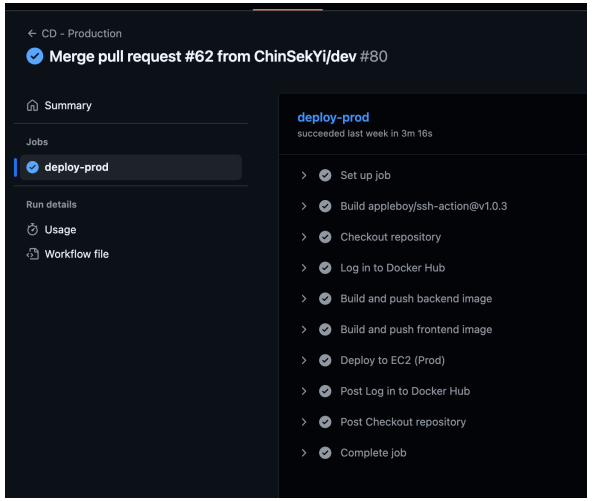


4.6.3 GitHub Actions View

CI	CD-Staging	CD-Prod



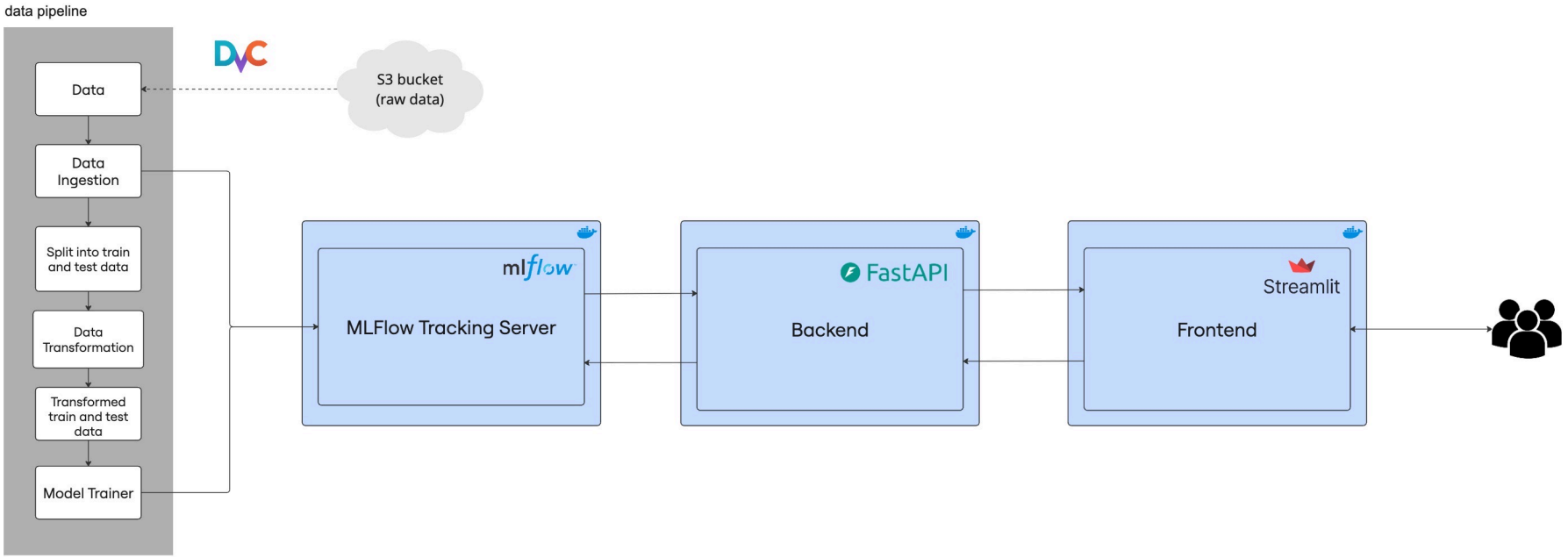


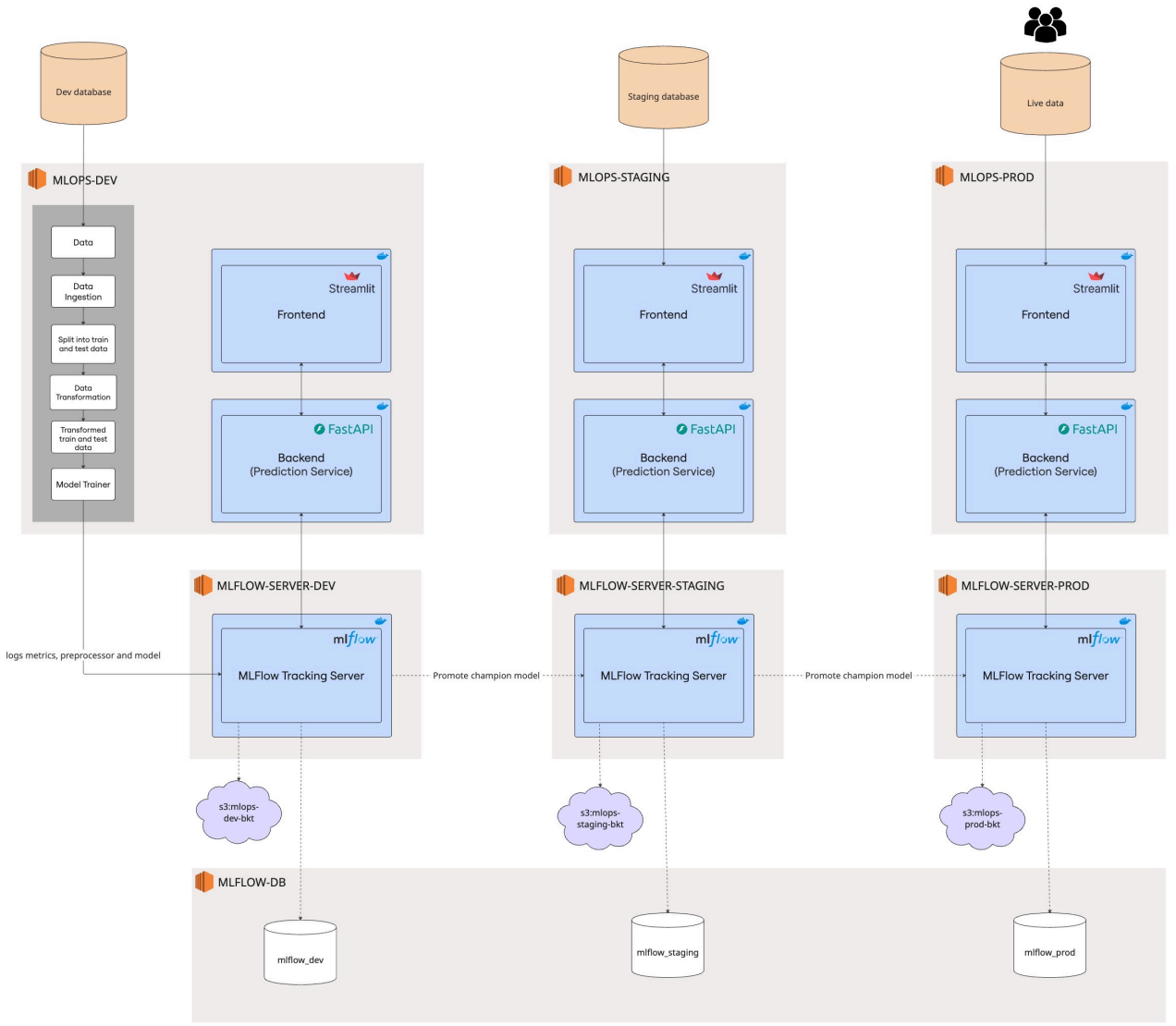


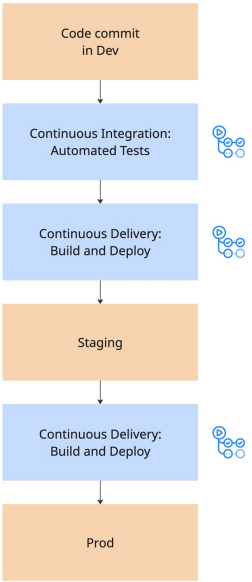
Roadmap

1. MLOps Motivation
2. System Overview & Use Case — Building the Foundation
3. Pipeline Component — The Data & Model Pipeline
4. Infrastructure & Deployment— Scaling Across Environments ✓

Demo


data pipeline
Data
Data Ingestion
Split into train and test data
Data Transformation
Transformed train and test data
Model Trainer
S3 bucket (raw data)
mlflow
MLFlow Tracking Server
FastAPI
Backend
Streamlit
Frontend

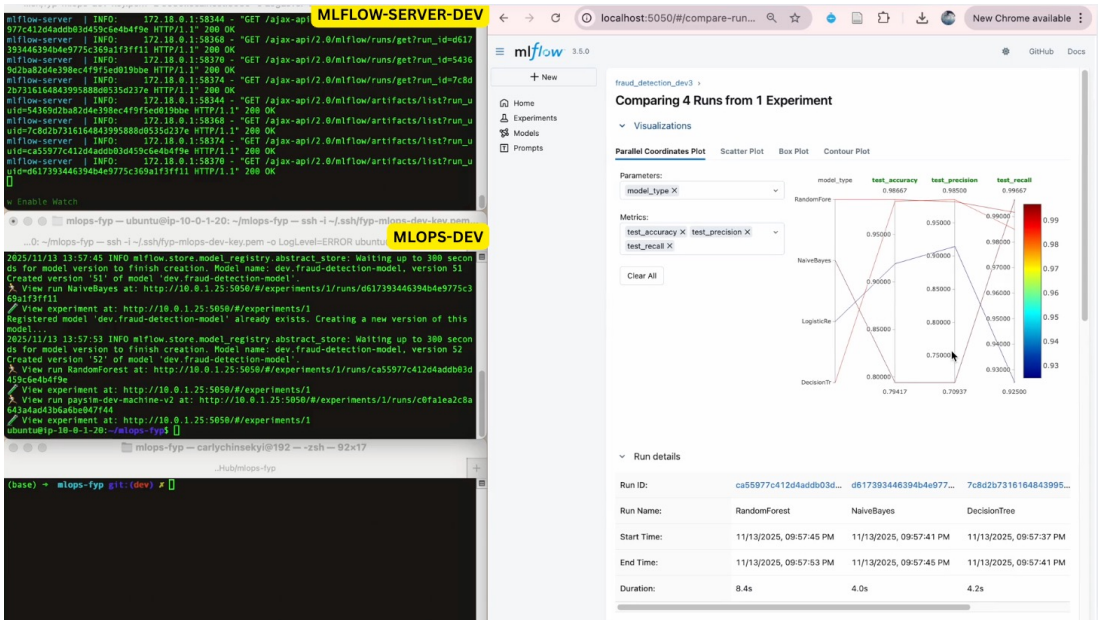

Dev database
Staging database
Live data
MLOPS-DEV
Data
Data Ingestion
Split into train and test data
Data Transformation
Transformed train and test data
Model Trainer
Streamlit
Frontend
FastAPI
Backend
(Prediction Service)
MLOPS-STAGING
MLOPS-PROD
MLFLOW-SERVER-DEV
MLFLOW-SERVER-STAGING
MLFLOW-SERVER-PROD
mlflow
MLFlow Tracking Server
logs metrics, preprocessor and model
Promote champion model
Promote champion model
s3:mlops-dev-bkt
s3:mlops-staging-bkt
s3:mlops-prod-bkt
MLFLOW-DB
mlflow_dev
mlflow_staging
mlflow_prod

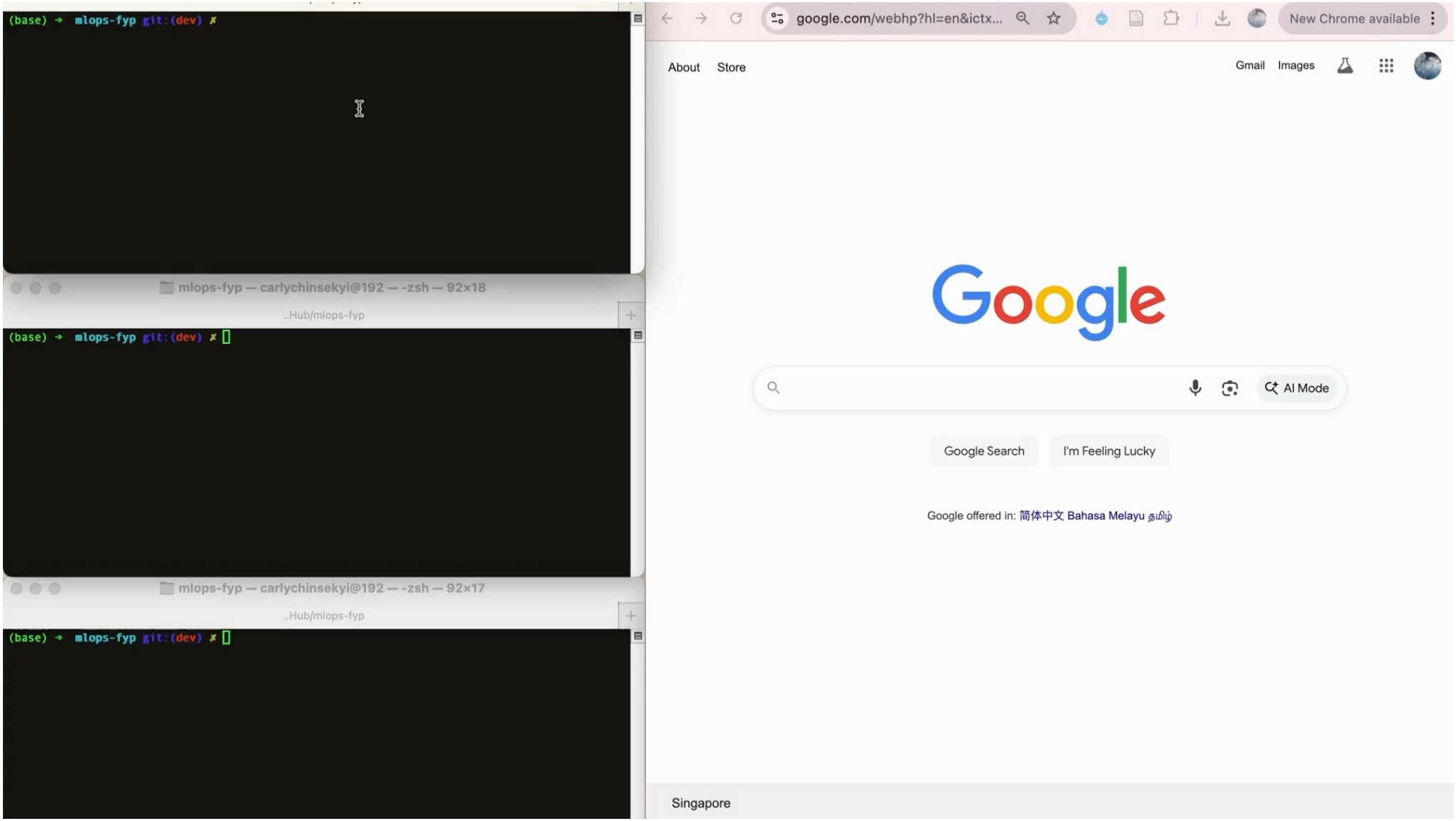

Code commit in Dev
Continuous Integration: Automated Tests
Continuous Delivery: Build and Deploy
Staging
Continuous Delivery: Build and Deploy
Prod

Demo 1:

Data Scientist's Development workflow

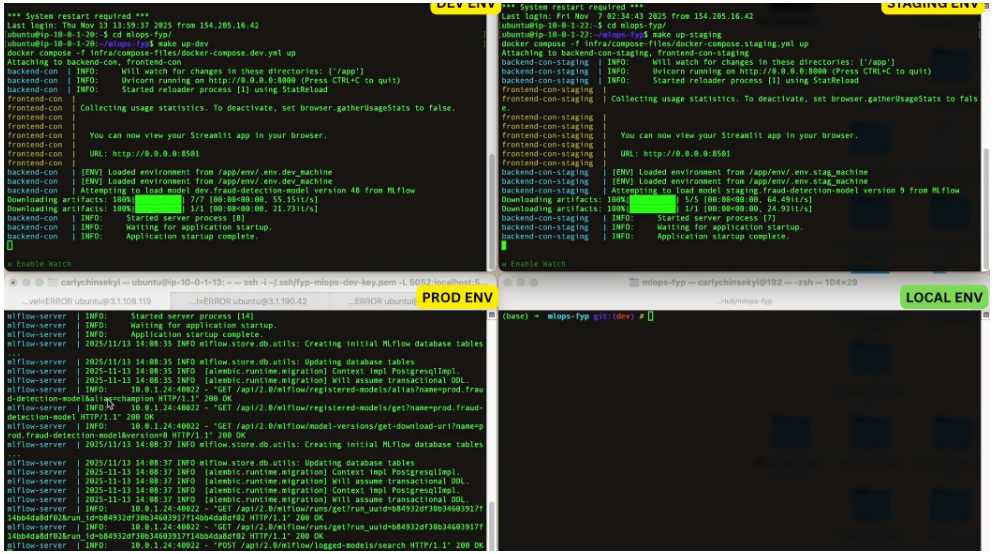
Data Scientist / ML Engineer

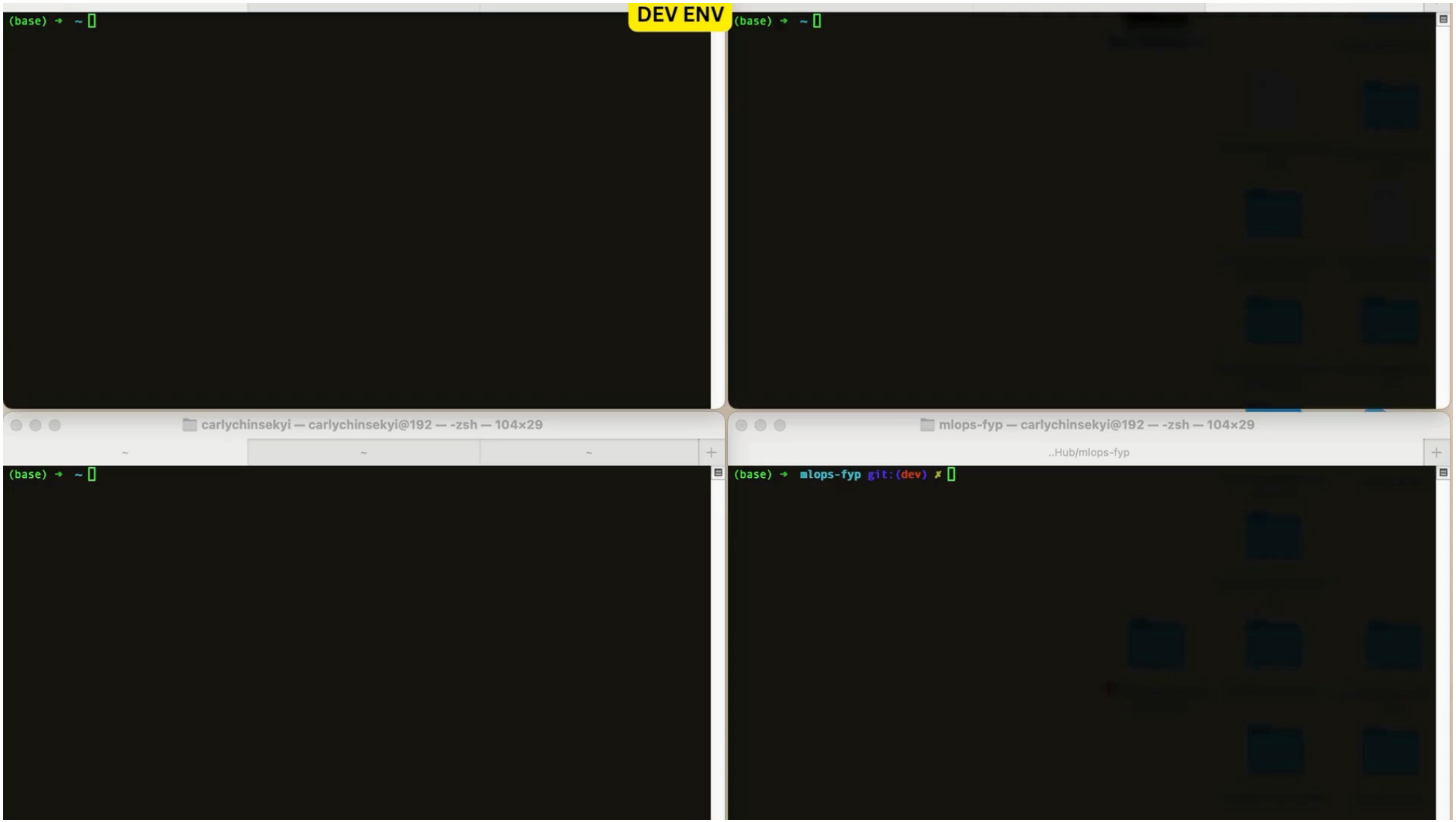



(base) ➜ mlops-fyp git:(dev) ✗
mlops-fyp — carlychinsekyi@192 — -zsh — 92×18
..Hub/mlops-fyp
(base) ➜ mlops-fyp git:(dev) ✗
mlops-fyp — carlychinsekyi@192 — -zsh — 92×17
..Hub/mlops-fyp
(base) ➜ mlops-fyp git:(dev) ✗
google.com/webhp?hl=en&ictx...
New Chrome available
About
Store
Gmail
Images
Google
AI Mode
Google Search
I'm Feeling Lucky
Google offered in: 简体中文 Bahasa Melayu தமிழ்
Singapore

Demo 2:

Dev, Staging and Prod Environment Simulation
+
Model Promotion Workflow

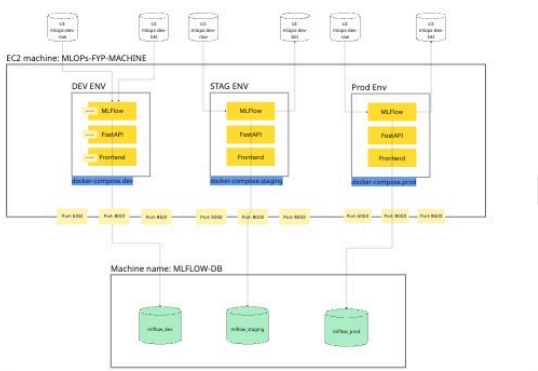



DEV ENV
(base) ➜ ~
(base) ➜ ~
carlychinsekyi — carlychinsekyi@192 — -zsh — 104×29
(base) ➜ ~
mlops-fyp — carlychinsekyi@192 — -zsh — 104×29
..Hub/mlops-fyp
(base) ➜ mlops-fyp git:(dev) ✗

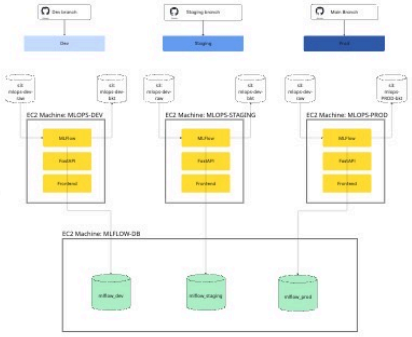
Evaluation

- **Development Velocity:**
  - Experiment setup time reduced from 5 hours to under 5 minutes (95% faster).
  - Model deployment became near-instant via MLflow registry.
- **Reproducibility & Collaboration:**
  - DVC + MLflow enabled full experiment reproducibility and seamless sharing.
  - Central MLflow server replaced error-prone manual tracking.
- **Error Reduction:**
  - Automated pipelines eliminated code/data/config integration issues.
  - Early bug detection via Docker builds and CI/CD validation.
- **Knowledge Transfer:**
  - Standardised pipeline lowered onboarding time for new team members.

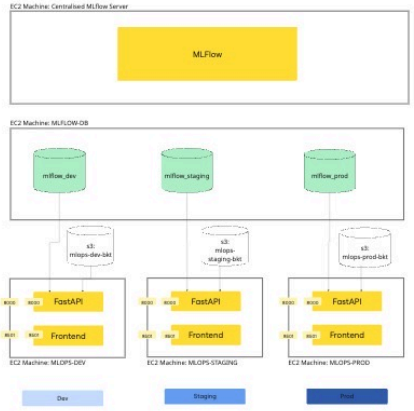
Project Evolution



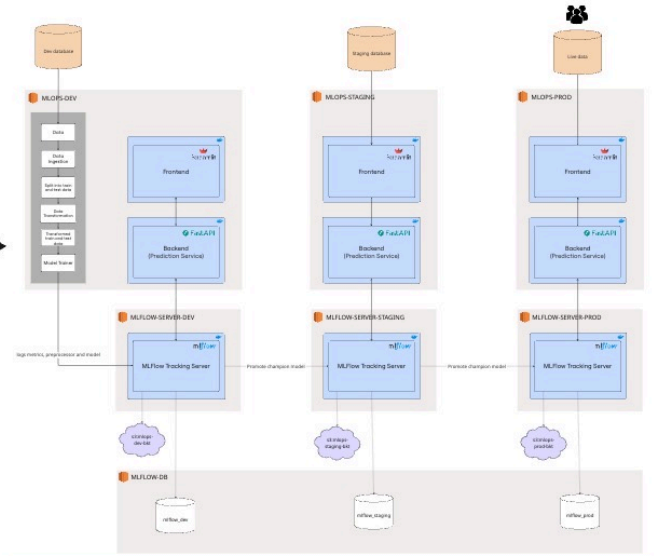
Plan A: Single EC2, Multi-Container



Plan B: Multi-EC2 Isolation



Plan C: Central MLflow Server



Plan D: Final Multi-Environment Setup

Challenges & Limitations

Challenges

- Setting up multi-environment AWS infrastructure.
- Docker and EC2 networking and connectivity between services required troubleshooting.
- MLflow configuration for artifact storage and experiment migration was tricky.
- CI/CD failures often requires many git commits to resolve.

Limitations

- No automated monitoring for data/model drift—manual checks only.
- CI/CD does not support automated retraining; retraining is manual.
- No feature store—features managed manually, limiting reusability.
- Several manual steps remain (retraining, model promotion); reducing these would improve reliability and scalability.

Future Work

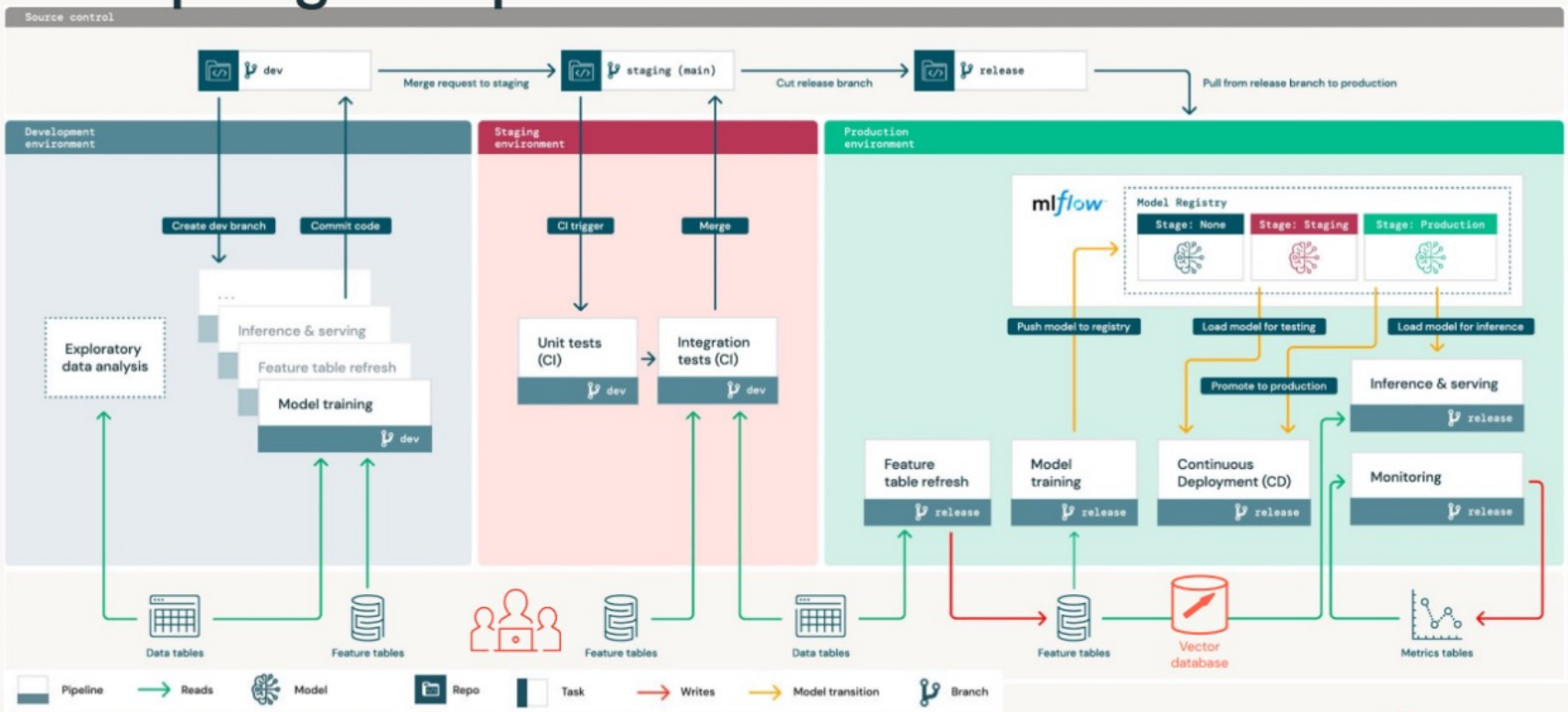
- Automated drift monitoring & alerting (Evidently AI, Grafana)
- CI/CD-triggered retraining & deployment (Jenkins)
- Centralized feature store for consistency & reuse
- Full automation of retraining & model promotion
- Auto-scaling for production workloads (Kubernetes)
- Support for deep learning frameworks (TensorFlow, PyTorch)
- Extend architecture for LLMOps & large language models

Conclusion

- Bridged the gap between academic ML and real-world impact
- Delivered a reproducible, production-ready MLOps platform
- Automated deployment, monitoring, and environment separation
- Enabled models to move from notebooks to serving real users
- Set the stage for future innovation (deep learning, LLMOps)

Adapting MLOps for LLMs

Some things change—but even more remain similar.



Thank You

Any Questions?

By: Chin Sek Yi

Project repository: https://github.com/ChinSekYi/mlops-fyp

Please try the Fraud Detection Model!

http://3.1.190.42:8501/